

University of Central Florida

**STARS**

---

Graduate Thesis and Dissertation post-2024

---

2025

# Autonomous Navigation and Real-Time 3D Reconstruction of Interior Spaces Using a Quadruped Robot

Steve Wilhelm

*University of Central Florida*

Find similar works at: <https://stars.library.ucf.edu/etd2024>

University of Central Florida Libraries <http://library.ucf.edu>

This Thesis is brought to you for free and open access by STARS. It has been accepted for inclusion in Graduate Thesis and Dissertation post-2024 by an authorized administrator of STARS. For more information, please contact [STARS@ucf.edu](mailto:STARS@ucf.edu).

---

## STARS Citation

Wilhelm, Steve, "Autonomous Navigation and Real-Time 3D Reconstruction of Interior Spaces Using a Quadruped Robot" (2025). *Graduate Thesis and Dissertation post-2024*. 389.  
<https://stars.library.ucf.edu/etd2024/389>

AUTONOMOUS NAVIGATION AND REAL-TIME 3D RECONSTRUCTION  
OF INTERIOR SPACES USING A QUADRUPEL ROBOT

by

STEVE WILHELM  
M.Sc., Bundeswehr University Munich, 2015

A thesis submitted in partial fulfillment of the requirements  
for the degree of Master of Science  
in the School of Modeling, Simulation and Training  
in the College of Engineering and Computer Science  
at the University of Central Florida  
Orlando, Florida

Summer Term  
2025

Major Professor: Dirk Reiners

## ABSTRACT

This thesis develops and evaluates a ROS2-based application that enables the quadruped robot Unitree Go2 to simultaneously perform autonomous navigation, real-time mapping, and 3D reconstruction in indoor environments using Simultaneous Localization and Mapping (SLAM). The robot system, equipped with onboard sensors, aims to generate high-resolution point clouds that provide detailed 3D scans. The assessment focuses on the platform's practical feasibility, operational reliability, and accuracy. The hardware setup combines the Unitree Go2 robot with its built-in sensors, a high-resolution 3D LiDAR, and an RGB-D camera. Software components include ROS2, the Unitree SDK, a navigation framework, and SLAM approaches. The application was developed using ROS-compatible open-source repositories for the sensors and SLAM algorithms, as well as Unitree's components and my customizations. Tests were conducted primarily in a university hallway and evaluated the quality of 3D reconstruction, computational load, wireless control capabilities, and navigation performance. The results show that the system generates dense, accurate 3D point clouds in real time, although mapping and reconstruction inaccuracies increase with longer or complex scans. Autonomous navigation based on 2D SLAM maps proved effective in simple environments but was sensitive to sensor quality. Running 2D and 3D SLAM simultaneously with live visualization requires high computational effort, risking degraded output without powerful external processing. Wireless operation is possible over short-range, high-bandwidth networks, but is limited under unstable conditions. The use of open source frameworks such as ROS2 and established SLAM algorithms facilitates application development but requires precise adaptation to the robot and the sensors used. The developed system demonstrates practical potential for real-world tasks such as indoor

exploration and reconnaissance. Future work should focus on implementing multi-sensor fusion SLAM, enhancing wireless communication, and integrating advanced navigation strategies. Combining multiple mobile robots could further enhance comprehensive mapping across diverse environments.

## **ACKNOWLEDGMENTS**

I would like to express my deepest gratitude to my professor and chair, Dr. Dirk Reiners, for his invaluable support and for making this thesis possible. His guidance and encouragement throughout the process were instrumental to the successful completion of this work. I would also like to extend my thanks to the other members of my committee for their insightful feedback and support. Additionally, I am grateful to the Unitree support team for their prompt and helpful responses to the numerous technical inquiries I had during my thesis.

Lastly, I would like to thank my family, especially my spouse and children, whose unwavering belief in me provided courage and motivation throughout this process, even during the most challenging times. I would also like to express my gratitude to my German comrade who stood by me with support and solidarity.

## TABLE OF CONTENTS

|                                                   |      |
|---------------------------------------------------|------|
| LIST OF FIGURES .....                             | viii |
| LIST OF TABLES .....                              | x    |
| LIST OF ABBREVIATIONS .....                       | xi   |
| CHAPTER ONE: INTRODUCTION.....                    | 1    |
| CHAPTER TWO: LITERATURE REVIEW .....              | 5    |
| Robot Localization, Navigation, and Mapping ..... | 5    |
| SLAM Fundamentals.....                            | 5    |
| Applied SLAM for Mobile Robots.....               | 9    |
| 3D Reconstruction Using Mobile Robots .....       | 13   |
| CHAPTER THREE: METHODOLOGY .....                  | 18   |
| Hardware Platform .....                           | 18   |
| Unitree Go2 EDU .....                             | 18   |
| Unitree 4D LiDAR L1 .....                         | 19   |
| Expansion Dock.....                               | 20   |
| Network Configuration .....                       | 22   |
| Hesai XT32 LiDAR.....                             | 23   |
| Intel RealSense Depth Camera D435i.....           | 25   |
| Software Frameworks .....                         | 26   |
| Unitree SDK .....                                 | 26   |
| ROS .....                                         | 26   |
| SLAM.....                                         | 28   |
| Application Development .....                     | 31   |

|                                                           |    |
|-----------------------------------------------------------|----|
| Sensor Packages .....                                     | 31 |
| Hesai LiDAR .....                                         | 31 |
| RealSense Camera .....                                    | 33 |
| Go2 Functionalities .....                                 | 33 |
| IMU and Odometry.....                                     | 34 |
| L1 LiDAR .....                                            | 34 |
| Other Functions.....                                      | 34 |
| 3D Reconstruction and Navigation .....                    | 36 |
| 3D SLAM .....                                             | 36 |
| 2D Mapping and Navigation.....                            | 39 |
| Sensor Configuration and Test Scenarios.....              | 40 |
| CHAPTER FOUR: RESULTS.....                                | 44 |
| Initial Observations .....                                | 44 |
| Real-Time 3D Reconstruction and Point Cloud Quality ..... | 45 |
| Simultaneous Mapping and Autonomous Navigation.....       | 52 |
| Wireless Operation .....                                  | 57 |
| Main Contributions .....                                  | 59 |
| CHAPTER FIVE: DISCUSSION.....                             | 62 |
| Findings and Implications .....                           | 62 |
| Limitations and Recommendations .....                     | 64 |
| Challenges During Implementation.....                     | 68 |
| CHAPTER SIX: CONCLUSION .....                             | 73 |
| APPENDIX A: EXPANSION DOCK RECOVERY AND UPDATE .....      | 75 |
| APPENDIX B: INITIAL EXPANSION DOCK SYSTEM SETUP .....     | 78 |

|                                                             |    |
|-------------------------------------------------------------|----|
| APPENDIX C: INSTALL USB WI-FI ADAPTER.....                  | 83 |
| APPENDIX D: SETTING UP ROS2 SUPPORT OF UNITREE ROBOTS ..... | 85 |
| APPENDIX E: DOCKER INSTALLATION ON GO2 EDU .....            | 87 |
| APPENDIX F: CONFIGURE EXPANSION DOCK FOR SLAM .....         | 90 |
| LIST OF REFERENCES.....                                     | 92 |



## LIST OF FIGURES

|                                                                                                                                          |    |
|------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 1: Overview of the SLAM algorithm procedure .....                                                                                 | 7  |
| Figure 2: Go2 EDU with top-mounted RGB-D camera, 3D LiDAR, and USB Wi-Fi adapter....                                                     | 19 |
| Figure 3: Expansion dock on top of Go2 EDU’s back.....                                                                                   | 20 |
| Figure 4: Internal layout of the Jetson Orin NX, with the NVMe SSD centered (marked in red)                                              | 22 |
| Figure 5: Hesai XT32 coordinate system shown in RViz .....                                                                               | 24 |
| Figure 6: Snapshot of the point cloud generated by the Hesai XT32 LiDAR, visualized in RViz<br>along with a model of the Go2 robot ..... | 32 |
| Figure 7: Snapshot of the point cloud generated by the Unitree L1 LiDAR, visualized in RViz.                                             | 35 |
| Figure 8: Launched user interface in RViz showing the map, robot dog, point cloud, SLAM<br>Toolbox, camera view, and control panel ..... | 43 |
| Figure 9: Real-time 3D scan of a hallway based on the filtered registered point cloud generated<br>by LIO-SAM.....                       | 46 |
| Figure 10: Real-time 3D scan of a hallway based on the “raw” registered point cloud generated<br>by LIO-SAM.....                         | 46 |
| Figure 11: Bird’s-eye view of the saved 3D map generated by LIO-SAM, visualized using<br>pcl_viewer.....                                 | 47 |
| Figure 12: Real-time 3D scan of a hallway based on the “raw” de-skewed point cloud generated<br>by DLIO .....                            | 48 |
| Figure 13: Bird’s-eye view of the saved 3D map generated by DLIO, visualized using pcl_viewer<br>.....                                   | 49 |
| Figure 14: Larger map generated by LIO-SAM, showing noticeable drifts in the point cloud on<br>the left side.....                        | 50 |
| Figure 15: High-angle view of the saved high-resolution point cloud scan .....                                                           | 50 |
| Figure 16: Real-time, colorized 3D scan showing the Go2 moving forward in the hallway.....                                               | 51 |
| Figure 17: Real-time generated 2D map of a hallway with the surrounding 3D point cloud.....                                              | 52 |
| Figure 18: Generated 2D map of a hallway alongside the 3D point cloud using the XT32 LiDAR<br>.....                                      | 54 |

|                                                                                                                                                                      |    |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| Figure 19: Generated map of hallway segment using the L1 LiDAR .....                                                                                                 | 54 |
| Figure 20: Navigation map; darker areas represent obstacles avoided during path planning and navigation.....                                                         | 55 |
| Figure 21: Comparison of the straight-line path between start and goal and the actual navigation path of the Go2, including an intermediate goal near the sofa ..... | 56 |
| Figure 22: Generated 2D and 3D maps showing shifts after sharp turns during autonomous navigation in a tight environment .....                                       | 57 |

## LIST OF TABLES

|                                                             |    |
|-------------------------------------------------------------|----|
| Table 1: Sensor configuration and operating parameters..... | 41 |
|-------------------------------------------------------------|----|

## LIST OF ABBREVIATIONS

|         |                                               |
|---------|-----------------------------------------------|
| AP      | Access Point                                  |
| BIM     | Building Information Model                    |
| CPU     | Central Processing Unit                       |
| DDS     | Data Distribution Service                     |
| DLIO    | Direct LiDAR-Inertial Odometry                |
| EOL     | End of Life                                   |
| FPS     | Frames per second                             |
| GPS     | Global Positioning System                     |
| GPT     | Generative Pre-trained Transformer            |
| GPU     | Graphics Processing Unit                      |
| ICP     | Iterative Closest Point                       |
| IMU     | Inertial Measurement Unit                     |
| IP      | Internet Protocol                             |
| LiDAR   | Light Detection and Ranging                   |
| LIO-SAM | LiDAR-Inertial Odometry-Smoothing and Mapping |
| LPDDR   | Low-Power Double Data Rate                    |

|          |                                             |
|----------|---------------------------------------------|
| LVI-SAM  | LiDAR-Visual-Inertial-Smoothing and Mapping |
| Mbps     | Megabits per second                         |
| MCU      | Motion Control Unit                         |
| NVMe     | Non-volatile Memory Express                 |
| OS       | Operating System                            |
| PC       | Personal Computer                           |
| PCD      | Point Cloud Data                            |
| QoS      | Quality of Service                          |
| RGB-D    | Red, Green, Blue-Depth                      |
| ROS      | Robot Operating System                      |
| RPM      | Revolutions per minute                      |
| RTAB-Map | Real-Time Appearance-Based Mapping          |
| RViz     | ROS Visualization                           |
| SDK      | Software Development Kit                    |
| SLAM     | Simultaneous Localization and Mapping       |
| SSD      | Solid State Drive                           |
| SSH      | Secure Shell                                |

|      |                                     |
|------|-------------------------------------|
| TOPS | Trillion Operations per second      |
| UGV  | Unmanned Ground Vehicle             |
| URDF | Unified Robotics Description Format |
| YAML | Yet Another Markup Language         |

## **CHAPTER ONE: INTRODUCTION**

3D reconstruction is used to generate 3D models of real-world objects or 3D scenes of indoor and outdoor environments. Areas of application include architecture, infrastructure inspection and construction, robotics, environmental monitoring, and reconnaissance (Gojda, 2023; J. Kim et al., 2022; Pepe et al., 2022; Xue et al., 2021). Today, Light Detection and Ranging (LiDAR) techniques, which rely on laser pulse scanning, are often used to collect 3D data, known as point cloud data. Depending on the application, data can be gathered using aircraft, drones, portable scanners, or unmanned ground vehicles (UGV).

Quadruped robots, a special form of UGVs, also called robot dogs, are particularly advantageous for scanning environments because they have higher degrees of freedom in motion control and can move through more complex, uneven terrain both inside and outside buildings (D. Hu et al., 2023). Famous quadruped robots feature Spot by Boston Dynamics and Laikago by Unitree Robotics. Key features of quadruped robots include integrated sensors such as LiDAR, camera, or Inertial Measurement Unit (IMU), onboard computer for real-time sensor processing, advanced motion control and terrain adaptation, and autonomous movement (Fan et al., 2024).

Applications of quadruped robots can be found in infrastructure inspection and monitoring, search and rescue missions, surveillance, agriculture, military reconnaissance, and other fields (N. Li et al., 2023; Q. Li et al., 2025; Tanveer et al., 2024; Wang et al., 2024).

Ongoing research on quadruped robots focuses on enhancing their performance by integrating advanced, multi-sensor technologies with real-time and cooperative control algorithms.

Specialized applications, such as disaster response, military reconnaissance, and environmental

monitoring, could particularly benefit from ground-air synergy enabled by collaboration with unmanned aerial vehicles, thereby improving task efficiency and safety (Fan et al., 2024).

In this work, the term “real-time” (or “online”) refers to the simultaneous processing of sensor data for 3D scanning, reconstruction, mapping, localization, and navigation while a quadruped robot is in motion. This process can also be visualized live and monitored in real time. In contrast, “offline” refers to the post-processing of previously recorded sensor data.

Quadruped robots have proven effective as mobile scanners for real-time 3D reconstruction of interior spaces, as they can autonomously navigate through interior spaces and generate detailed 3D point clouds. (Gan et al., 2025; D. Hu et al., 2024; Y. Li & Shi, 2018; Rezende et al., 2020). Localization and mapping are essential components of robot navigation, as they allow robots to interpret their environment and accurately estimate their position (Qiao et al., 2024). Simultaneous Localization and Mapping (SLAM) techniques address these challenges by utilizing integrated sensors to iteratively build a map while simultaneously estimating the robot’s position within it (Yarovoi & Cho, 2024). This approach provides a solid foundation for real-time tracking and the creation of 3D scenes of interior spaces (Gan et al., 2024). Although the use of 3D point cloud data from laser scanning and SLAM algorithms for real-time localization and 3D reconstruction with quadruped robots represents a promising solution, research on this topic remains limited, with ongoing studies focusing particularly on multi-sensor fusion and the use of advanced techniques like deep learning for point cloud processing, aimed at improving the accuracy and reliability of navigation and 3D scene reconstruction (D. Hu et al., 2023; D. Hu & Gan, 2025).



The objective of this thesis is to develop an application that enables a quadruped robot to perform autonomous navigation while simultaneously conducting real-time mapping and 3D reconstruction of indoor environments. The system leverages multiple sensor modalities, including a 3D LiDAR, an IMU, and odometry data, to achieve accurate localization and high-fidelity environmental perception. The aim is to generate a high-resolution point cloud representing a detailed 3D scan of the surroundings. As part of this process, the thesis examines under which conditions and with which limitations such a system can be implemented reliably, accurately, and practically. Particular attention is given to the ease of development, the availability and suitability of software frameworks and tools, hardware requirements, and the practical challenges encountered during implementation. For this purpose, the quadruped robot Unitree Go2 EDU (Unitree Robotics, 2025i), equipped with integrated sensors, a compute module, and additional 3D LiDAR and RGB-D camera, will be utilized. The broader goal is to assess the feasibility of such a system for real-world applications, such as reconnaissance and surveillance scenarios, potentially in collaboration with unmanned aerial vehicles.

The contribution of this work lies in the development of a fully integrated, modular ROS2-based system that enables simultaneous 3D reconstruction and autonomous navigation on a quadruped robot. Unlike most previous research, which often treats 3D mapping and navigation as separate tasks, performs 3D reconstruction offline, relies on pre-built maps for navigation, or uses wheeled robots (e.g., Gan et al., 2024; Pang et al., 2024), this work demonstrates real-time operation on a terrain-adaptive quadruped platform equipped with a high-resolution LiDAR and RGB-D camera. Furthermore, the system fuses multi-modal sensor data using multiple SLAM techniques to achieve robust and accurate 2D and 3D mapping, while also

enabling the saving of high-resolution 3D scans during operation for further processing and analysis. In addition, this work provides a practical evaluation of the development complexity, integration challenges, and system limitations under real-world indoor conditions.

The rest of this thesis paper is structured as follows. Chapter 2 presents a review of related work, covering current research on robot localization, mapping, and navigation, as well as 3D scanning approaches with mobile robotic platforms. Chapter 3 describes the methodology and technical implementation, including the setup of the mobile robot system and the sensors, the selection of software frameworks and tools used to develop the application, and the design of test scenarios. Chapter 4 presents the results obtained from the test scenarios, and Chapter 5 discusses the findings along with their implications, limitations, and challenges encountered during implementation. Finally, Chapter 6 summarizes the key findings, reflects on the feasibility of the developed approach, and provides an outlook on potential future applications.

## CHAPTER TWO: LITERATURE REVIEW

This chapter provides an overview of existing research related to robot localization, navigation, mapping, and 3D reconstruction. The review is structured into two parts: The first part focuses on techniques and frameworks for localization and navigation in robotics, while the second part examines methods for 3D reconstruction using mobile robot platforms.

### Robot Localization, Navigation, and Mapping

To scan interior spaces, create maps, and provide 3D point cloud reconstruction, UGVs must be capable of navigating through environments while simultaneously determining their position. Robot-based scanning systems that rely on GPS information are often unsuitable for indoor use due to weak signal support (Gan et al., 2024). Furthermore, traditional navigation systems typically rely on pre-built maps to plan routes and localize the robot within a known environment (Vashisht et al., 2025). However, these systems are often limited in complex, dynamic, or unknown settings. Instead, SLAM algorithms are used, which iteratively build a map of the environment using the integrated sensors (mapping) and estimate the pose of the robot or device within the map (localization) (Yarovi & Cho, 2024).

### SLAM Fundamentals

The SLAM problem is a fundamental problem in mobile robotics. A robot must construct a map of an unknown environment and simultaneously estimate its own pose within this map. Formally, SLAM can be expressed as the problem of estimating the joint posterior distribution over the robot's trajectory  $x_{1:T}$  and the environment map  $m$ , given sensor observations  $z_{1:T}$ ,

control inputs or odometry readings  $u_{1:T}$ , and an initial pose  $x_0$  (Durrant-Whyte & Bailey, 2006; Grisetti et al., 2010). In this formulation, sequences of all variables from time step  $t = 1$  to  $T$  are considered:

$$p(x_{1:T}, m | z_{1:T}, u_{1:T}, x_0) \quad (1)$$

The goal is to estimate both the robot path and the map, despite inherent uncertainties due to sensor noise, odometry drift, dynamic environments, and ambiguous observations. Because pose and map estimations are interdependent – pose estimation errors corrupt the map, and map inaccuracies affect localization – SLAM is fundamentally a nonlinear estimation problem (Kok et al., 2024).

To address this challenge, SLAM is typically approached with recursive Bayesian estimation methods that update beliefs over time as new control and sensor data become available. The joint posterior over the robot pose  $x_k$  and the map  $m$ , given all observations  $Z_{0:k}$  and control inputs  $U_{0:k}$  up to time  $k$ , can be computed recursively through two main steps: prediction and correction, also known as update step (Durrant-Whyte & Bailey, 2006). In the prediction step, the prior distribution over the robot’s state is estimated using the motion model:

$$p(x_k, m | Z_{0:k-1}, U_{0:k}, x_0) = \int p(x_k | x_{k-1}, u_k) p(x_{k-1}, m | Z_{0:k-1}, U_{0:k-1}, x_0) dx_{k-1} \quad (2)$$

In the correction step, the latest observation  $z_k$  is incorporated using the observation model. Here,  $\eta$  is a normalizing constant ensuring the distribution sums to one:

$$p(x_k, m | Z_{0:k}, U_{0:k}, x_0) = \eta p(z_k | x_k, m) p(x_k, m | Z_{0:k-1}, U_{0:k}, x_0) \quad (3)$$

Figure 1 illustrates the SLAM estimation cycle, based on a diagram by Abdulmajeed and Mansoor (2014), showing how the motion and observation models work together to recursively update the robot's pose and map.

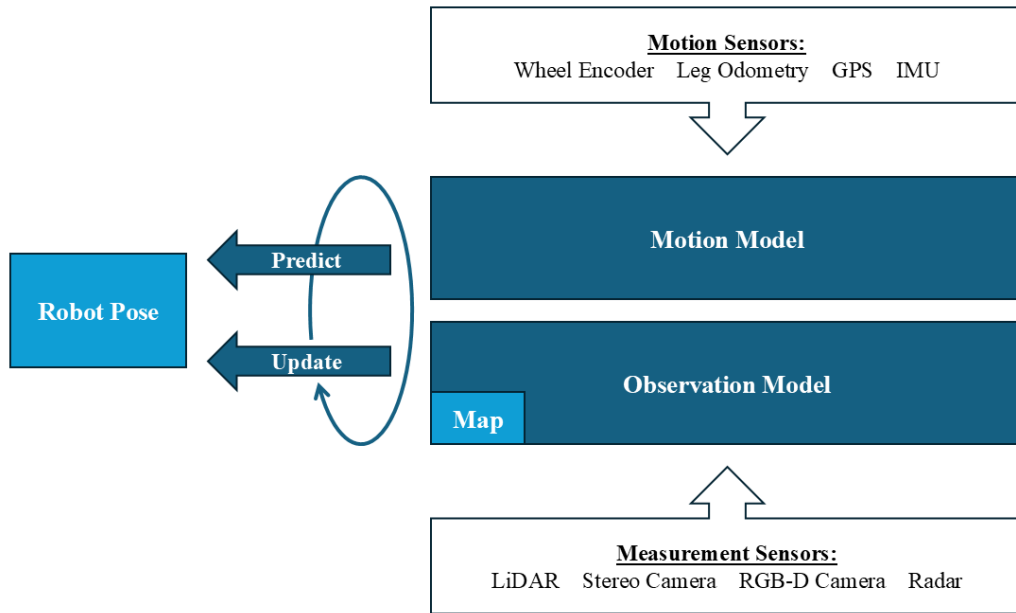


Figure 1: Overview of the SLAM algorithm procedure

The motion model predicts the probability of the robot's next pose given its previous pose and the applied control input (motion sensors), while the observation model corrects the predicted pose using data from measurement sensors, such as observed features or grid cells, and updates the environment map accordingly (Durrant-Whyte & Bailey, 2006). This recursive Bayesian model forms the basis of many SLAM algorithms and enables the simultaneous estimation of the robot trajectory and the environment map despite noisy sensors and uncertain motion.

Since the exact calculation of these distributions is generally difficult, practical methods rely on approximations. Classical approaches include Extended Kalman Filter SLAM, which linearizes motion and observation models and maintains a Gaussian estimate over the joint state of robot pose and map landmarks (distinct and observable features in the environment, such as corners or edges, used for data association), or particle filter-based methods, which represent the posterior with a set of weighted particles, each maintaining an individual map hypothesis (Durrant-Whyte & Bailey, 2006). Modern approaches increasingly favor graph-based SLAM, which represents robot poses as nodes and spatial constraints (e.g., scan matches or odometry) as edges in a graph. This formulation minimizes the error across the pose graph through nonlinear optimization, thus enabling scalable and accurate map construction (Kümmerle et al., 2011).

In LiDAR-based SLAM systems, mapping is performed by extracting geometric features or registering dense point clouds generated from laser scans. Feature-based SLAM detects stable, repeatable landmarks, such as planes, edges, or corners, within each scan and uses them to establish correspondences across time for motion estimation and mapping. In contrast, dense methods register raw or filtered point clouds using algorithms such as Iterative Closest Point

(ICP; Besl & McKay, 1992) or scan-to-submap alignment and often utilize incremental optimization to improve accuracy (J. Zhang & Singh, 2014).

A critical challenge in LiDAR SLAM arises from the continuous acquisition of scans: As the sensor moves during a scan sweep, points can be spatially distorted due to motion. To address this, LiDAR-inertial SLAM methods incorporate IMU data to perform motion correction or de-skewing, compensating for robot movement within the scan duration (Shan et al., 2020). This fusion of LiDAR and IMU data enables accurate pose estimation even during rapid or complex maneuvers, mitigating the effect of motion-induced distortions and improving the fidelity of the resulting point cloud maps.

The map is constructed incrementally by transforming each corrected scan into a global coordinate frame using the estimated trajectory. Detecting loop closures – recognizing previously visited locations – allows the SLAM system to introduce additional constraints to the pose graph and optimize globally, significantly reducing accumulated drift over long trajectories (Grisetti et al., 2010). In legged mobile robots, where wheel odometry is often unavailable or unreliable, integrating IMU data with LiDAR scans is particularly advantageous. This tightly coupled sensor fusion enables robust SLAM performance in complex, unstructured indoor environments where GPS is unavailable (Shan et al., 2020).

### **Applied SLAM for Mobile Robots**

Building on the theoretical foundation of SLAM, a wide range of practical systems and sensor configurations have been developed to enable robust localization, mapping, and autonomous navigation in complex environments. The creation of the map and accurate

localization of a robot is crucial for autonomous robot navigation (Z. Chen et al., 2025).

Nowadays, there are a variety of SLAM algorithms and approaches for mobile ground robots based on different sensor and data types, including LiDAR, monocular, stereo, and RGB-D cameras, odometry data from motion sensors, linear acceleration and angular velocity from an IMU, or a fusion of these sources (Lou et al., 2023; Rezende et al., 2020; Zhu et al., 2024). Existing SLAM methods for localization and navigation are largely based on 2D laser scans, such as GMapping (Grisetti et al., 2007) or Cartographer (Hess et al., 2016), but 3D SLAM systems, for example based on 3D LiDAR scans or sensor fusion, are also frequently used for this purpose and are currently the subject of increasing research (Z. Chen et al., 2025).

In an experimental study, Li and Shi (2018) developed a simple indoor mapping and autonomous navigation program using a mobile robotic platform and a SLAM algorithm based on a particle filter. Their mobile robot system was based on a low-cost, high-performance hardware platform with modular components, including a single-line LiDAR and an RGB-D camera as the main sensors for environmental sensing. The robot's software system was developed using the open-source Robot Operating System (ROS) (Open Robotics, 2021), enhancing the modularity, portability, and ease of development of the software. Their experiments demonstrated the robot's ability to create accurate 2D maps and navigate autonomously indoors, confirming the effectiveness of the robot system. Rezende et al. (2020) compared several pose estimation algorithms based on wheel, visual, and LiDAR odometry, and ultra-wideband radio signals, all fused with IMU data for indoor localization and autonomous navigation. They used a mobile robot system running ROS and various sensors, including a top-mounted 16-channel LiDAR, encoders, and a tracking camera. Their experiment showed that



LiDAR odometry and SLAM provided the best results for controlling indoor robots and were prerequisites for autonomous navigation. Pang et al. (2024) developed a low-cost SLAM system design for autonomous navigation and real-time mapping in complex indoor environments. They adopted the LiDAR-inertial odometry framework FAST-LIO (W. Xu & Zhang, 2021) and incorporated the Fast ICP algorithm (Rusinkiewicz & Levoy, 2001) for precise point cloud alignment and mapping, as well as 3D autonomous exploration capabilities, and confirmed accuracy and efficiency even in a lower-cost SLAM system.

Today, research in the field of multi-sensor fusion essentially aims to improve the ability of efficient and safe autonomous robot navigation in complex and unknown environments (Sun et al., 2024). Xu et al. (2024) presented a multi-sensor fusion SLAM approach with object detection for quadruped robots, which they extensively evaluated in simulation environments. Their sensor fusion framework, based on LiDAR, RGB-D camera, and IMU, and leveraging visual and LiDAR odometry data, utilized advanced methods such as the combination of two SLAM algorithms, LIO-SAM (Shan et al., 2020) and LVI-SAM (Shan et al., 2021), and the use of neural network-based object detection, enabling precise map generation and accurate target tracking. Sun et al. (2024) also demonstrated through their experiments that a navigation framework consisting of multiple SLAM sensors, such as LiDAR, RGB-D camera, and IMU, improves the autonomous ability of a mobile ground robot in indoor and complex outdoor environments.

Digital 3D Building Information Models (BIM) are also increasingly being used in architecture and construction to support 3D SLAM in indoor spaces (Torres et al., 2023). Chen et al. (2025) developed a BIM-aided navigation and localization framework for autonomous robotic

inspections using quadruped robots. Their robotic platform consisted of the quadruped robot Unitree Go1, an RGB-D camera with built-in IMU, and a 2D LiDAR scanner. Using this multi-sensor SLAM system, they combined inertially corrected 2D laser scans with fused laser and visual-inertial SLAM, achieving collision-free navigation and a significant reduction in localization drift compared to traditional SLAM methods. Zhai et al. (2024) also used a BIM-enabled approach to optimize quadruped robot navigation in interior spaces.

In real-time applications, it is particularly important that mobile ground robots can navigate safely, especially in dynamic environments. Obstacle detection and avoidance are therefore essential for autonomous navigation. Standard navigation systems for mobile robots, such as Nav2 (Macenski et al., 2020), generally offer the capability for dynamic obstacle avoidance. However, research continues to improve these functions, such as the stable detection and prediction of obstacles in unstructured environments (Jian et al., 2023). For instance, Dai et al. (2024) proposed a novel formulation of point cloud control barrier function (Ames et al., 2019) and local planner to navigate robots safely in real-world scenarios. In their experimental study, they used the quadruped robots Unitree B1 and Unitree Go2, equipped with a 16-channel LiDAR and an NVIDIA Jetson Orin compute module, to test their novel approach. Using the SLAM Toolbox (Macenski & Jambrecic, 2021) for 2D mapping and localization, they successfully tested the effectiveness of their navigation approach in a real-world scenario using only point cloud data. Likewise, Unlu et al. (2024) developed a motion planning algorithm based on a control barrier function framework for navigation and collision avoidance in unknown environments. Using LiDAR data for mapping and localization, they verified their effective navigation approach in an indoor scenario using the quadruped robot Unitree B1. C. Li et al.

(2024) proposed a robust visual SLAM approach (based on a stereo RGB camera) for small-scale quadruped robots for accurate localization in dynamic environments. Their SLAM system used visual and IMU-fused leg odometry data and integrated object detection and semantic segmentation, which allowed them to reduce positioning errors in dynamic scenes.

Given the diversity of SLAM approaches, each algorithm has its strengths and limitations, also depending on the sensor technology used (Qiao et al., 2024; Yarovoi & Cho, 2024; Zhu et al., 2024). In most setups with quadruped robots, a combination of LiDAR, camera, IMU, and odometry sensors was used for successful localization, mapping, and navigation in indoor environments. The development of custom robot sensor configurations is facilitated by the abundance of ROS-compatible, open-source SLAM algorithms and software repositories that can be adapted to specific needs and scenarios (Kanster, 2018; KeeganRen, 2018; T. Kim, 2023; Labbé, 2024; T. Lin, 2022; SilenceOverflow, 2023). A robust robot navigation and mapping framework forms the foundation for accurate 3D reconstruction of environments.

### **3D Reconstruction Using Mobile Robots**

The use of robot-assisted mobile scanning systems with integrated sensors and 3D SLAM techniques enables promising real-time 3D scanning in both indoor and outdoor environments (Kang et al., 2020; Pang et al., 2024). Mobile robotic platforms can be used to create 3D mapping and 3D reconstruction primarily by using LiDAR point cloud data or visual and depth images in combination with SLAM approaches (Kang et al., 2020). By combining multiple sensors on mobile robots, such as LiDAR and camera, the quality of 3D reconstruction can be significantly improved (Lou et al., 2023). In addition to multi-sensor fusion, advanced

techniques, such as deep learning methods for 3D point cloud processing, are being employed to enhance the 3D reconstruction process with mobile robots (Guo et al., 2021). There are a variety of open-source software repositories with deep learning approaches for 3D reconstruction and 3D point cloud segmentation methods that can be used together with mobile robots and SLAM algorithms (Q. Hu, 2021a, 2021b; Natowi, 2023; Yin, 2024).

Building inspections often involve the use of mobile robots, some of which are equipped solely with 3D LiDAR, alongside advanced 3D scanning and processing techniques (Ge & Sadhu, 2025; D. Hu & Gan, 2025). As a specialized type of mobile ground robots, quadruped robots are being used more frequently for this purpose. J. Kim et al. (2021, 2022) proposed a deep learning-based 3D semantic segmentation from LiDAR point cloud data of a scaffold by a quadruped dog. They used a Unitree A1 robot and a top-mounted IMU and LiDAR. They successfully acquired point cloud data of the scaffold with various scanning postures using the IMU- and LiDAR-equipped robot dog, mapped the point cloud data using the SLAM algorithm LIO-SAM (Shan et al., 2020), and used the deep learning encoder-decoder network RandLA-Net (Q. Hu et al., 2020) for performing semantic segmentation of the point cloud data. Although their approach is not suitable for indoor use, it shows that the robot dog can collect precise point cloud data with the connected LiDAR, process it using the SLAM algorithm, and create 3D reconstructions. Zhai et al. (2024) successfully captured a large portion of the building structure by using their BIM-based approach to optimize scan planning, combined with the Lidar\_slam2 method (Sasaki, 2022) for 3D scanning, which provides an offline reconstruction that allows for processing of all current and past observation data on LiDAR sensor pose and the map (Gan et al., 2025). This demonstrated the effectiveness of automated 3D reconstruction using a 3D

LiDAR sensor integrated into a quadruped robot. Chen et al. (2025) also achieved similarly good results in point cloud reconstruction using their multi-sensor SLAM and BIM-based approach with the Unitree Go1 robot. Zhang et al. (2025) proposed a new SLAM system based on LiDAR-inertial-camera fusion and a multimodal multilayer matching algorithm applied to a quadruped inspection robot. The robot used a 3D solid-state LiDAR with built-in IMU, a 3D mechanical LiDAR, and a monocular industrial camera for mapping, navigation, and localization, as well as a dual camera for image acquisition. Their results demonstrated a high-precision, robust SLAM approach with consistent 3D point cloud map generation, enabling the robot to reliably perform inspection tasks in various indoor and outdoor scenarios.

Some studies used multiple LiDAR sensors for navigation and 3D scanning. D. Hu et al. (2023) presented a new robot-assisted mobile scanning approach for 3D reconstruction and LiDAR point cloud segmentation of building interiors. They used the quadruped robot Unitree Go1 with a top-mounted 2D LiDAR sensor for navigation, an additional solid-state LiDAR sensor for 3D scan, and SLAM with robot motion control, pathfinding, and RGB-D image collection to obtain more complete and accurate point cloud data for 3D mapping and reconstruction of the indoor scene. They also proposed an enhanced dynamic window approach to combine the motion control with SLAM to estimate the pose of the LiDAR sensor more accurately, resulting in a dense 3D scene with higher quality. Lastly, they established a new robot pathfinding method to identify optimal scan positions and used the deep-learning method ResPointNet++ (Yin et al., 2021) for automated point cloud segmentation. To perform the scanning process, the robot navigates to a designated scanning position and rolls its body in various directions to scan the upper parts of the target wall and floor plate. When the legged

robot finishes scanning at a scanning position, it navigates to the next position and repeats the scanning process until all target walls are scanned. With their research, they were able to successfully confirm the practical use of legged robots for 3D reconstruction and the use of deep learning-based point cloud segmentation. Following this, D. Hu et al. (2024) used this scanning approach in developing a novel domain adaptation network to realize BIM-to-Scan point cloud semantic segmentation. In their study, they also used a robot-assisted scanning system consisting of the Unitree Go1 with an RGB-D camera and 2D LiDAR to acquire SLAM-generated point clouds of interior spaces. The RGB-D camera was used to collect high-resolution RGB and depth images, which were then converted into point clouds by a SLAM algorithm. The 2D LiDAR was used for the navigation of the robot dog. The point cloud data collected with the robot-assisted scanning system was used to effectively validate the performance of the domain adaptation network and train it. Gan et al. (2024) introduced an automated as-built 3D reconstruction of building interiors with a quadruped robot and a 3D LiDAR sensor. They integrated the LiDAR SLAM algorithm with robotic control to achieve detailed 3D/2D mapping and complete point cloud data acquisition. In their study, they proposed two 3D scanning solutions: one that couples 3D reconstruction and 2D mapping for robot navigation using only the 3D LiDAR sensor, and another that decouples 3D reconstruction and 2D mapping using an additional 2D LiDAR sensor dedicated exclusively to robot navigation. Despite some limitations in scanning positions, the quadruped robot's low-angle viewpoint, and the influence of light and transparency, they achieved promising results with the robot dog and the 3D LiDAR sensor regarding the completeness and density of point cloud data and suggested using separate 3D and 2D LiDAR sensors for independent 3D reconstruction and 2D mapping as the preferred solution. Building

on this, Gan et al. (2025) conducted further experiments on the automated 3D reconstruction approach, focusing on optimizing the view angle, robot trajectory, and obstacle avoidance. As before, the GMapping approach (Grisetti et al., 2007) was used for 2D mapping, and Lidarslam\_ros2 (Sasaki, 2022) was used to generate 3D point clouds. The results again demonstrated the efficiency of their decoupled, lightweight navigation and scanning method with the Unitree Go1 robot, even though they explicitly pointed out the limited memory and computing capacity of the robot dog's embedded system-on-module.

Although there are already promising approaches for 3D scanning using mobile ground robots, real-time 3D reconstruction, especially of indoor spaces, using quadruped robots equipped with LiDAR using SLAM and autonomous navigation continues to be explored (D. Hu et al., 2023). Dynamic environments, such as the presence of moving objects or fluctuating lighting conditions, pose challenges for autonomous navigation and mapping, as SLAM algorithms may struggle to adapt to these changes (Qiao et al., 2024). Robots can also encounter problems because they rely on embedded computers with limited memory and processing power. This poses a challenge for real-time processing, as the hardware may not be able to cope with the increased computational demands (Gan et al., 2025). Previous research also suggests that future studies should focus more on sensor fusion technologies to enable more accurate robot pose estimation, as well as advanced scanning and data processing techniques to improve SLAM systems and the precise generation of 3D point clouds (Gan et al., 2024; Lou et al., 2023; Rezende et al., 2020).

## **CHAPTER THREE: METHODOLOGY**

This chapter describes the hardware and software components used in this thesis, the application development, and the testing procedures performed to evaluate the robot system. It begins with an overview of the platform hardware, including the Unitree Go2 robot and the various sensors integrated into the system. The software frameworks used, such as the Unitree SDK, ROS, and SLAM algorithms, are then explained in detail. Following this, the chapter outlines the application development process, focusing on the key packages and functionalities implemented for 3D reconstruction and autonomous navigation. Finally, the sensor configurations and test scenarios for evaluating the system's indoor scanning and navigation performance are presented.

### **Hardware Platform**

#### **Unitree Go2 EDU**

The Unitree Go2 EDU is a highly advanced quadruped robot launched by Unitree Robotics in July 2023. The robot dog integrates state-of-the-art technologies, sensors, and features, such as high-performance motors, AI capabilities with GPT support, a position tracking module, advanced motion control modes, an intelligent side-tracking system, a super-wide-angle 3D LiDAR (4D LiDAR L1), Bluetooth 5.2, Wi-Fi 6, and a remote control with a range of 30 m (Unitree Robotics, 2025i). It is available in three versions – AIR, PRO, and EDU – each with different hardware features and capabilities, tailored to different user needs. The configured EDU version used in this thesis is shown in Figure 2.





*Figure 2: Go2 EDU with top-mounted RGB-D camera, 3D LiDAR, and USB Wi-Fi adapter*

Unitree Go2 uses Data Distribution Service (DDS), a middleware based on a publish/subscribe model that supports various Quality of Service (QoS) policies to ensure reliable real-time communication between its sensors, computing units, and control systems (Unitree Robotics, 2023a). The robot and its functions are primarily controlled via a mobile app but can also be implemented and used with the Unitree SDK and ROS. This app is also required for initial setup, calibration, and system updates (Unitree Robotics, 2025d).

### ***Unitree 4D LiDAR L1***

The integrated Unitree 4D LiDAR L1 is mounted on the bottom of Go2's head, providing  $360^\circ \times 90^\circ$  hemispheric ultra-wide-angle detection and a maximum scanning distance of 30 meters with an effective scan rate of 21,600 points/s. This allows the robot dog to create a real-time 3D image of its surroundings, navigate its environment, and avoid obstacles (Unitree Robotics, 2025a). The built-in LiDAR application, that is also accessible via a mobile app,

includes a LiDAR point cloud, an altitude map, and a 3D voxel map service that leverages point cloud and the robot's odometer to obtain the position and posture information of the robot dog, create a real-time visualization of the environment map centered around the robot, and capable of tracking the robot's movement and constantly updating the 3D map (Unitree Robotics, 2023b).

### ***Expansion Dock***

The EDU version of the Unitree Go2 features a powerful compute module mounted on top of its back, see Figure 3. The NVIDIA Jetson Orin NX 16GB module, which is equipped on the Go2 in this thesis, delivers up to 100 TOPS of performance in AI workloads, features an 8-core CPU, consumes between 10 to 25 watts depending on the operating mode, supports advanced video encoding and decoding capabilities, and includes 16 GB of LPDDR5 memory (Unitree Robotics, 2025c).



*Figure 3: Expansion dock on top of Go2 EDU's back*

The expansion dock allows for the connection of additional devices and sensors and manages their data processing. Typical devices that can be connected include a USB Wi-Fi adapter, a user PC connected via an RJ45 Ethernet port, and a USB docking station via USB-C port, to which a monitor can be connected via HDMI, and a wireless mouse and keyboard. Go2 EDU can be equipped by default with the Intel RealSense D435i RGB-D camera and an additional Hesai XT16 3D LiDAR or Livox Mid-360 3D LiDAR, which can be mounted on top of the Go2 and connected to the module. For the robot setup in this thesis, I used the Hesai XT32 LiDAR instead of the XT16 because its higher resolution and point density enhance the accuracy and detail of SLAM, mapping, and 3D reconstruction.

At the core of the external compute module is a compact NVMe SSD (256 GB, M.2 2242), as shown in Figure 4, providing internal storage for secondary development. The NVMe SSD includes the docking station’s firmware and the Ubuntu OS, which can be overwritten with the factory image if necessary, see APPENDIX A: EXPANSION DOCK RECOVERY AND UPDATE (Unitree Robotics, 2025c). Regular updates are necessary to enhance additional functionalities, such as the integrated SLAM and odometry services. The integrated SLAM and navigation services interface only supports EDU robots with an expansion dock and the Mid-360 or XT16 LiDAR (Unitree Robotics, 2025j). However, individual functionalities, such as the *dog\_control* node for providing robot IMU and odometry data, can be used independently.

For application development directly on the Go2, a monitor, mouse, and keyboard can be connected via a USB-C docking station, or remote access can be established using the NoMachine remote desktop software. Before starting application development, an initial setup of

the extension dock should be performed on the Ubuntu OS, see APPENDIX B: INITIAL EXPANSION DOCK SYSTEM SETUP.



*Figure 4: Internal layout of the Jetson Orin NX, with the NVMe SSD centered (marked in red)*

### ***Network Configuration***

For application development and program execution, the Go2 EDU must be connected to the user PC via Ethernet by default. Once connected, the IP address of the user PC must be set to 192.168.123.xxx (except for the IPs mentioned below) to enable communication with the Go2. The IP address of the Go2's expansion dock is 192.168.123.18. Using this address, the user can access the extension dock via SSH or remote desktop software (hostname: unitree, password: 123). In addition, the IP 192.168.123.161 is reserved for the Go2's motion control unit (MCU)

(Unitree Robotics, 2025j). The IP 192.168.123.20 is provided by default for the Mid-360 and Hesai XT16 LiDAR and is also used for the XT32 in this thesis.

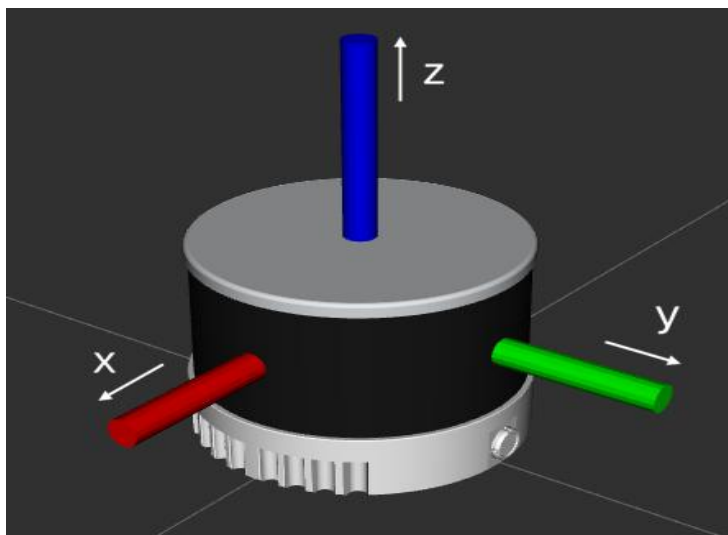
Although the Go2 has a Wi-Fi and Bluetooth interface used to communicate with the mobile app, secondary application development via the internal Wi-Fi interface is not possible. For this reason, if developers want to control Go2 via Wi-Fi with their applications or update software on the expansion dock, a wireless network interface, such as a Linux-compatible USB Wi-Fi adapter, must be connected to the docking station. To download suitable Wi-Fi drivers for the initial setup, a smartphone can establish a network connection via USB tethering (see APPENDIX C: INSTALL USB WI-FI ADAPTER). For my thesis, I connected a standard USB Wi-Fi adapter to the Go2 robot. The adapter supports a combined speed of up to 1800 Mbps, with a maximum of 1201 Mbps on the 5 GHz band and 574 Mbps on the 2.4 GHz band.

While ROS topics can be published over Wi-Fi using a USB Wi-Fi adapter, the Go2's internal ROS topics are not transmitted over Wi-Fi by default. Enabling this requires implementing and launching custom ROS nodes that access the data via the Unitree SDK (Unitree Robotics, 2025n, 2025o).

## **Hesai XT32 LiDAR**

The Hesai XT32 is a mechanical 32-channel mid-range LiDAR with a horizontal field of view of 360° and a vertical field of view of 31° (-16° to +15°) (Hesai Technology Co., Ltd., 2025c). It is a high-precision sensor that offers an instrumented range of 0.05 to 120 m, a point rate of up to 1,280,000 points/second, accuracy of  $\pm 1\text{--}2$  cm, and range precision of 0.5-2 cm. This enables high-resolution point cloud data. In addition to point cloud data, it can also provide

GPS data, which is not used in this thesis. The Hesai XT32 LiDAR offers adjustable horizontal resolution based on frame rate:  $0.09^\circ$  at 5 Hz,  $0.18^\circ$  at 10 Hz, and  $0.36^\circ$  at 20 Hz. It supports both single- and dual-return modes with options such as “last”, “strongest”, and “first” returns. In single-return mode, it delivers up to 640,000 points per second at a data rate of up to 22.52 Mbps, and in dual-return mode, up to 1,280,000 points per second at up to 45.04 Mbps. Frame rates are selectable between 5, 10 (default), and 20 Hz (Hesai Technology Co., Ltd., 2025c). The LiDAR is connected to Go2’s expansion dock via an M8 connector. A web interface (default via 192.168.1.201 in the browser; in this configuration via 192.168.123.20) is used to set parameters, display device information, and update software/firmware. Figure 5 shows the coordinate system underlying the LiDAR. The top-mounted Hesai XT32 uses a coordinate frame with +X pointing left, +Y backward, and +Z upward, which differs from the default ROS and Go2 robot frames, where +X is forward, +Y is left, and +Z is up, and must be considered during implementation.



*Figure 5: Hesai XT32 coordinate system shown in RViz*

Hesai's software repositories (Hesai Technology Co., Ltd., 2025b) provide a C++ SDK and ROS driver for the XT32 LiDAR, making it compatible for robot application development. PandarView 2 is the included point cloud visualization software that records and displays live point cloud data from the Hesai LiDAR (Hesai Technology Co., Ltd., 2024).

### **Intel RealSense Depth Camera D435i**

The Intel RealSense D435i is a compact, USB-powered depth camera that combines active stereo depth sensing with an integrated IMU. It features depth and RGB sensors, a wide field of view ( $87^\circ \times 58^\circ$  for depth,  $69^\circ \times 42^\circ$  for RGB), and an ideal sensing range of 0.3 to 3 m (Intel Corporation, 2025a). The D435i delivers a resolution of up to  $1280 \times 720$  pixels at up to 90 FPS for depth data, and  $1920 \times 1080$  pixels at 30 FPS for RGB. The camera is supported by the open-source Intel RealSense SDK 2.0 and a ROS wrapper that allows integration into robotics applications (Intel Corporation, 2025b). The camera uses the depth sensor as the coordinate system origin, with all other sensor data transformed into it (Intel Corporation, 2023). The camera should be connected to the designated USB 3.2 Type-C interface of the Go2 expansion dock to ensure optimal data transfer performance. The Go2 EDU also features an odometer services interface for the D435i. It provides high-speed semi-direct visual odometry capable of generating motion trajectories (Unitree Robotics, 2024c).

## Software Frameworks

### Unitree SDK

The Unitree SDK, *unitree\_sdk2*, provides developers with a comprehensive toolkit to create advanced applications for the Go2 and other Unitree robots. The SDK includes interfaces for low-level motor control, high-level motion planning, LiDAR point cloud processing, audio and video streaming, SLAM, odometry, and provides relevant function interfaces (Unitree Robotics, 2024b). Go2 leverages DDS (*Cyclone DDS*) as its messaging middleware, enabling real-time, scalable communication between the functional modules, sensors, cloud services, the mobile app, user PC, and the external compute module (Unitree Robotics, 2024a). DDS supports both C++ and Python and is fully compatible with ROS2. Unitree provides a detailed SDK development guide outlining all the necessary steps to set up secondary development, especially available software interface services for application development (Unitree Robotics, 2025b). In addition, Unitree offers open-source software repositories that support development using C++, Python, and ROS (Unitree Robotics, 2025n, 2025p, 2025o).

### ROS

ROS is an open-source framework designed to support software development for robotic systems. It provides developers with a standardized software platform with a set of software libraries, tools, algorithms, and conventions that simplify the development of robotic applications (Open Robotics, 2021). While ROS1 was preferred for many development and research projects in the past, its improved successor, ROS2, is now the preferred choice for new projects. ROS2 also uses DDS as middleware, which ensures compatibility with the Unitree SDK and allows



direct communication and control of Unitree robots without the need to wrap the SDK interface (Unitree Robotics, 2024e).

There are different ROS distributions (versioned sets of ROS packages). By default, Go2 EDU's Ubuntu OS supports application development with the latest ROS1 distribution, ROS Noetic (EOL May 31st, 2025), and ROS2 Foxy (EOL June 20th, 2023), with all Go2 topics published via ROS2. Software development with the newer ROS2 versions, ROS2 Humble and Jazzy, is also possible from a connected user PC or by using a Docker container directly on the Go2 itself. In this thesis, ROS2 Humble was used for development.

In ROS2, application development is structured around modular software units called nodes, each responsible for specific tasks such as sensor processing or path planning. Nodes can publish or subscribe to topics, such as point cloud or IMU data, offer or call services, and use actions for long-running tasks. Applications are organized into packages containing source code, dependencies (*package.xml*), and build instructions (*CMakeLists.txt* or *setup.py* for Python). These packages are built in a workspace using the *colcon* build system. The launch and execution of nodes are managed via Python-based launch files, which allow for parameter configuration and the orchestration of multiple nodes across one or more machines. Configuration parameters are often stored in YAML files that are loaded into the launch process to specify node behavior or sensor settings. Communication between nodes uses DDS middleware, which manages message transport over the network and allows the configuration of network adapters and QoS settings. In addition, ROS2 leverages the *tf2* library (Foote, 2013) to manage coordinate transformations between different reference frames, which are essential for tasks such as localization, sensor fusion, and navigation. These frames represent coordinate

systems associated with various parts of the robot or its environment, such as *base\_link* (robot base), *odom* (odometry), *map* (global reference), or sensor-specific frames such as *camera\_link* (camera) or *lidar\_link* (LiDAR). Nodes can broadcast and listen to transformations, enabling consistent tracking of robot and sensor positions across the system over time. For visualizing sensor data, robot models, and status information, there is the ROS 3D robot visualizer RViz (or RViz2 in ROS2), a visualization tool with a 3D graphical interface. Developers can use it to display data such as LiDAR point clouds, camera images, robot trajectories, coordinate systems (using *tf2*), and navigation paths.

The development and execution in ROS2 follow a structured, exemplary process: After sourcing the ROS2 distribution with `$ source /opt/ros/humble/setup.bash`, packages can be built with `$ colcon build`. The workspace is sourced via `$ source install/setup.bash`. To set up ROS 2 support for the Unitree robot (see APPENDIX D: SETTING UP ROS2 SUPPORT OF UNITREE ROBOTS), the command `$ source unitree_ros2/setup.sh` is executed, and nodes are launched using either `$ ros2 run my_package my_node` for single nodes or `$ ros2 launch my_package my_launch_file.py` for multiple nodes via launch files. If it is not already started by the launch file, RViz can be called via `$ rviz2`, and the necessary configuration for visualization can be loaded.

## SLAM

In my thesis, I utilize three different SLAM frameworks. SLAM Toolbox is a 2D SLAM system based on laser scanning and odometry data, designed for both online and offline mapping and is particularly effective in larger indoor spaces (Macenski & Jambrecic, 2021). It is the

default SLAM application in ROS2. The *slam\_toolbox* (Macenski, 2025c) package is open-source and available in every current ROS distribution. SLAM Toolbox leverages graph-based optimization by using laser scan and odometry data to generate pose-scan pairs, create a pose graph to compute robot pose, and simultaneously create an accurate 2D map of the environment. In my work, SLAM Toolbox performs real-time 2D mapping and localization using 2D laser scan data converted from either the Hesai XT32 or Unitree L1 LiDAR point clouds, along with odometry data generated by a 3D SLAM system. SLAM Toolbox is fully compatible with the Nav2 navigation stack (Macenski et al., 2020), the ROS2 Navigation framework for autonomous navigation, localization, and path planning, which I use in my thesis, and that enables mobile robots to navigate through complex environments while avoiding obstacles. Both SLAM Toolbox and Nav2 provide RViz plugins that allow users to save or load generated maps and subsequently perform localization and navigation tasks.

For real-time 3D reconstruction, I use a 3D SLAM system. In my thesis, I tested two different 3D SLAM approaches, DLIO and LIO-SAM. DLIO is a modern, lightweight direct LiDAR-inertial odometry algorithm that couples LiDAR scans with IMU, using a coarse-to-fine approach to construct continuous trajectories for precise motion distortion correction (K. Chen et al., 2023). It features a compact architecture that combines motion correction and prior construction in a single step with direct scan-to-map registration, supported by a robust nonlinear geometric observer for accurate state estimation, enabling accurate localization and 3D mapping while maintaining low computational overhead. Extensive testing, including with Hesai LiDARs, has confirmed its algorithmic efficiency, making it well-suited for real-time robotics applications. In my setup, DLIO utilizes point cloud data from the Hesai XT32 LiDAR and IMU data from the

Unitree Go2 to perform real-time 3D SLAM. DLIO supports ROS1 by default, but there is also a ROS2 branch available (Verifiable & Control-Theoretic Robotics Laboratory, 2024).

LIO-SAM is a framework for tightly-coupled LiDAR-inertial odometry via smoothing and mapping that employs a factor graph-based optimization approach to fuse LiDAR and IMU data (Shan et al., 2020). The algorithm combines LiDAR and IMU data into a global factor map to estimate the robot’s trajectory. It uses raw IMU measurements to de-skew point clouds and provide motion estimates for LiDAR odometry optimization. The resulting LiDAR odometry is further used to refine the IMU bias estimation. Instead of global scan matching, LIO-SAM performs local scan registration with a sliding keyframe window. This improves real-time performance and scalability, achieving high accuracy and robustness in 3D environments. LIO-SAM supports ROS1 by default, but a ROS2 branch is also available for integration with ROS2 environments (Shan, 2023). Due to its algorithmic design, LIO-SAM is more closely tied to LiDAR-specific characteristics. While it was originally tested extensively with Velodyne and Ouster LiDARs, recent adaptations have extended its compatibility to other LiDAR types, such as Hesai LiDAR sensors (X. Hu, 2025). In my work, LIO-SAM also uses the point cloud data from the Hesai XT32 LiDAR and the IMU data from the Unitree Go2. To improve localization accuracy, the odometry generated by LIO-SAM is fused with the internal odometry data provided by the Go2 robot itself. In addition to testing LIO-SAM with the Hesai LiDAR, I also explored sensor fusion with the RGB-D camera by assigning color values from the RGB images to the corresponding points in the LiDAR-generated point cloud to enhance the visual quality of the point cloud.

## Application Development

Several software repositories were used to develop the navigation and 3D scanning application. The application is designed to run both directly on the Go2 and a user PC connected to it. The software packages are built using ROS2 Humble. Since the Go2 natively supports only ROS2 Foxy, the application must be launched in a Docker container on the robot, providing an encapsulated platform with a full ROS2 Humble setup. For the Go2 EDU, NVIDIA Docker is also required to access the Jetson Orin module's GPU within the container. See APPENDIX E: DOCKER INSTALLATION ON GO2 EDU for details. The entire application is made available to the University of Central Florida as an internal GitHub repository.

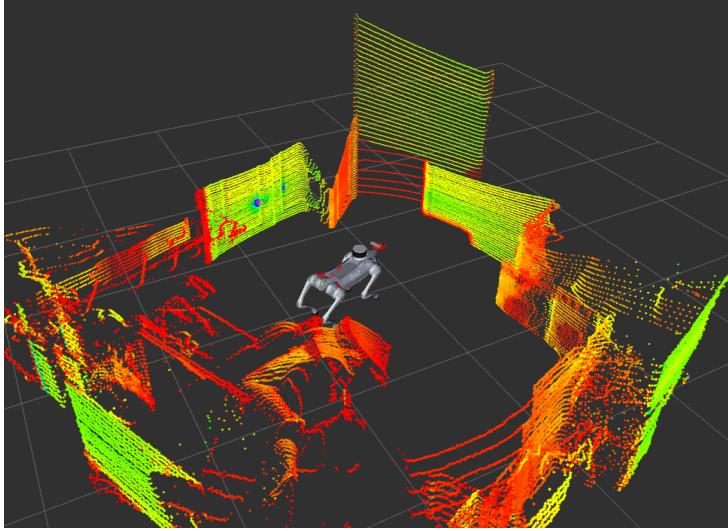
### Sensor Packages

The application of SLAM approaches for 3D scanning and autonomous navigation requires the availability of sensor data. This includes LiDAR, inertial, odometry, and visual (camera) data.

#### *Hesai LiDAR*

Hesai Technology Co., Ltd (2025a) offers a software repository with the ROS2 driver for Hesai LiDAR. It parses LiDAR data and publishes point cloud frames into a ROS topic. The package must be adapted to the respective LiDAR type. Accordingly, the configuration for this thesis must be modified to include the XT32, the LiDAR IP (192.168.123.20), the provision of system timestamps, and, if necessary, the frame and topic names (default ROS2 frame ID: *hesai\_lidar*, topic: */lidar\_points*). *PointCloud2* is the name of the message type of point cloud topics in ROS2. In addition, settings such as the LiDAR frequency can be set in the LiDAR web

interface. Figure 6 shows the point cloud of a small room, generated by the Hesai XT32 LiDAR and visualized in RViz after launching the Hesai ROS2 driver.



*Figure 6: Snapshot of the point cloud generated by the Hesai XT32 LiDAR, visualized in RViz along with a model of the Go2 robot*

LiDAR data can be extensive, especially in high-resolution 3D scanning applications, depending on the LiDAR type used (e.g., channel number, scan frequency). This can be particularly problematic if communication with the Go2 is wireless via Wi-Fi connections with insufficient bandwidth. For this reason, only compressed LiDAR data should be transferred between Go2 and the user PC when bandwidth is low. *Point\_cloud\_transport* is a ROS package for subscribing to and publishing *PointCloud2* messages over different transport layers and can provide support for transporting point clouds in low-bandwidth environments (Papllham, 2019). For this purpose, several compression libraries are available as plugins (Cordero, 2024). For this thesis, the *zlib\_point\_cloud\_transport* plugin, based on the *zlib* library (Adler, 2025) and

utilizing the deflate compression method, proved useful due to its favorable compression ratio and high decompression quality.

### ***RealSense Camera***

Intel provides the ROS Wrapper for Intel RealSense cameras, which enables the use of RealSense cameras with ROS2 (Intel Corporation, 2024). *Camera\_link* is the name of the ROS2 main frame of the camera. For this thesis, I use the camera's RGB sensor to enable real-time live view and support sensor fusion. RGB image data is published via the */camera/color/image\_raw* topic in ROS2. The RealSense ROS2 driver automatically provides the necessary transformations between the main frame and the RGB and depth sensor frames. The camera package, which is integrated directly into ROS2 after installation, allows sensor settings to be specified either via the launch command, e.g., by setting the RGB profile to 640×480 resolution at 30 FPS, or by loading a parameter configuration file. The RealSense camera node must be launched on the Go2 itself. It cannot be started from a connected user PC. Since camera data can be exceptionally large, image compression is recommended for low-bandwidth Wi-Fi connections to ensure efficient data transmission. Compressed topics are automatically provided via the RealSense ROS Wrapper, which can be enabled by installing the *image-transport-plugin* package (Cordero, 2025a).

### **Go2 Functionalities**

Some additional sensor data from the Go2 is processed in a customized *go2\_driver* node. The *go2\_driver* node subscribes to the robot's IMU, odometry, and LiDAR topics and republishes them under different names with updated timestamp for use in SLAM applications.

## ***IMU and Odometry***

The most important sensor data that the Go2 itself must provide is IMU data for the SLAM approaches. Although Go2 EDU's built-in SLAM and navigation services interface cannot be used with the XT32 LiDAR, it includes the *dog\_control* node, which publishes the robot's IMU (*/dog\_imu\_raw*) and odometry (*/dog\_odom*) topics. If this node is launched directly on the Go2, the data can be used for the SLAM applications.

## ***L1 LiDAR***

Since the 2D mapping is also evaluated with the integrated Unitree L1 LiDAR, the corresponding LiDAR topic is required. When the Go2 starts, the L1 LiDAR is automatically started, and the raw point cloud is published as a ROS2 topic */utlidar/cloud*. If the application runs over Wi-Fi, the *point\_cloud\_transport* package can also be used to compress the L1 point cloud. Figure 7 shows a snapshot of the point cloud data recorded over the past 3 seconds of a small room, generated by the Unitree L1 LiDAR and visualized in RViz.

## ***Other Functions***

To enable both manual and autonomous navigation for the Go2 robot via ROS, a command velocity node (*cmd\_vel*) must be available. *Cmd\_vel* for Go2 is implemented in the *go2\_driver* node. The *cmd\_vel* topic is responsible for controlling the robot's movement by specifying linear and angular velocities. A *Twist* message is published to this topic to initiate and control the robot's motion. To simplify manual control of the Go2 via the keyboard, the *teleop\_twist\_keyboard* package (Lalancette, 2024) integrated in ROS2 can be used. Alternatively, a customized RViz plugin is available that allows the Go2 to be moved using the mouse.



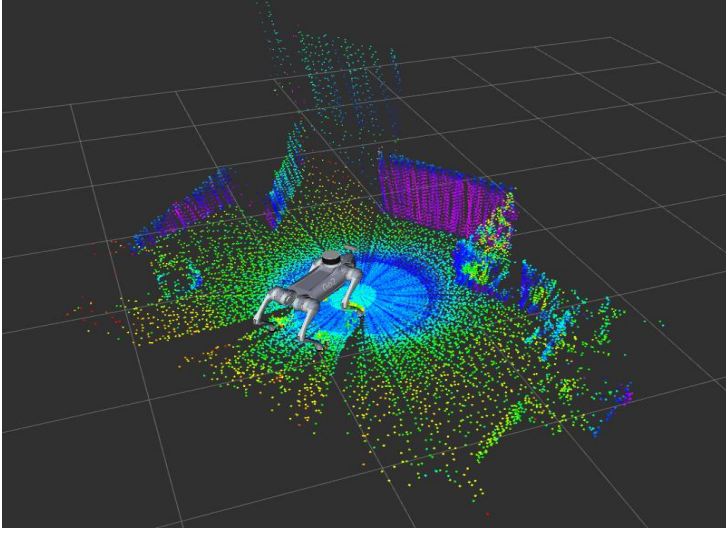


Figure 7: Snapshot of the point cloud generated by the Unitree L1 LiDAR, visualized in RViz

To visually represent the Go2 in RViz, a so-called URDF file with the kinematic tree model of the robot is required. The model with the standard Go2 robot description, provided by Unitree (Unitree Robotics, 2025e), was extended to include the Hesai LiDAR and the RealSense camera. In addition, Go2's *joint\_states* are published by the *go2\_driver* node. These messages report the current motor state or position and velocity of each joint in the robot, such as the front right hip joint. Along with the URDF model, this joint data is used by the *robot\_state\_publisher* package (Cordero, 2025b), which is integrated into ROS2, to compute and broadcast the robot's coordinate frames (*tf2*). This enables accurate visualization and tracking of the robot's posture in RViz.

Furthermore, the packages *unitree\_go* and *unitree\_api*, as well as the Cyclone DDS setup script from the *unitree\_ros2* package (Unitree Robotics, 2025n), are required for communication with the Go2 and are therefore included as part of this application.

### **3D Reconstruction and Navigation**

For 3D reconstruction, the 3D SLAM approaches LIO-SAM (Shan et al., 2020) and DLIO (K. Chen et al., 2023), adapted to the Go2 and the Hesai LiDAR XT32, are implemented. For 2D mapping, SLAM Toolbox (Macenski & Jambrecic, 2021) is used, providing the map required for autonomous navigation using the ROS2 navigation framework and system (Macenski et al., 2020, 2023).

### ***3D SLAM***

To implement DLIO, I used the official ROS2 branch of the algorithm (Verifiable & Control-Theoretic Robotics Laboratory, 2024). To adapt to the sensor data of the Go2, the corresponding IMU topic of the robot dog and the point cloud topic of the Hesai XT32 were defined in the launch file. To calibrate the sensors, the extrinsic values between the robot's base frame and the LiDAR, as well as between the base frame and the IMU, had to be set in the DLIO configuration file. The information and calculations for the corresponding translation vector and rotation matrix can be found in the documentation of the SLAM and navigation services interface (Unitree Robotics, 2025j), as well as in the robot description (Unitree Robotics, 2025e). Once the configuration is adjusted, the SLAM application can be launched. After mapping is complete, the generated map can be saved in PCD file format (used for storing point clouds with coordinates and features) and further processed. These files can be viewed, for example, using the *pcl\_viewer* tool. In addition to real-time processing with the SLAM algorithm, the raw IMU and LiDAR data can be stored using *rosvbag2*, a tool for recording and playback of ROS2 topics

(Knese, 2025). This enables the SLAM application to be run offline and allows further processing of the raw data.

To implement LIO-SAM, I started with the official ROS2 branch of the algorithm (Shan, 2023). Since LIO-SAM is LiDAR-specific, the point cloud data must be formatted correctly for cloud de-skewing, primarily in the *imageProjection.cpp* file of the LIO-SAM package. Because the coordinate system of the Hesai XT32 (see Figure 5) and the timestamps of its point clouds deviate from the standard, these adjustments must be implemented in the source code, as demonstrated by X. Hu (2025). Furthermore, initial tests with the Go2 have shown that the IMU pre-integration in the LIO-SAM package, which enables accurate motion estimation between LiDAR scans by efficiently accumulating IMU data, does not produce sufficiently reliable IMU odometry, with large drift (errors in the robot’s estimated position) observed in the SLAM visualization. It should be noted that Unitree also uses LIO-SAM with the XT16 in its SLAM and navigation services interface (Unitree Robotics, 2025j), which requires adjustments to the IMU and odometry for compatibility with LIO-SAM. However, these adjustments are not open source and are therefore not publicly available. For this reason, the odometry handler in the *imuPreintegration.cpp* file of the LIO-SAM package is removed. In addition, the odometry data from the Go2, provided by the *dog\_control* node, serves as an additional input to LIO-SAM.

To compensate for the missing odometry handler and improve movement of the Go2 within LIO-SAM, I used a modified *odom\_to\_tf\_ros2* package (Okolo, 2025; Stavrinou, 2025) to fuse the two sources of odometry: the high-accuracy but low-rate odometry from LIO-SAM and the low-accuracy but high-rate odometry from the *dog\_control* node. This fusion generated an

odometry transform with both high accuracy and a high update rate, enabling faster and more stable movement of the Go2 robot.

In the LIO-SAM configuration file, the LiDAR, IMU, and odometry topics, as well as the associated LiDAR frame and robot base frame of the Go2 platform, must be specified. In addition, for the XT32 LiDAR, the sensor type, number of rings, horizontal resolution, and LiDAR range, as well as the extrinsic values between the robot's IMU and the LiDAR, must be set. Furthermore, depending on indoor or outdoor use, the voxel filter parameters must be adjusted in the configuration file. With these adjustments, the SLAM application can be started. After mapping is complete, the generated map is automatically saved in various versions in PCD file format. Raw data from IMU, odometry, and LiDAR can also be saved with rosbag2. In addition, I implemented a *point\_cloud\_accumulator* node that can be used to store all high-resolution, de-skewed point clouds from LIO-SAM or DLIO into a PCD file.

With LIO-SAM, I aimed to perform sensor fusion with the RGB-D camera by using image data to assign color values to the corresponding points in the LiDAR point cloud. For this purpose, the camera image data is processed in the *mapOptimization.cpp* file of the LIO-SAM package, where it is matched with the LiDAR point cloud, and the corresponding RGB values are applied, as demonstrated by Zeren (2023). The camera topics must be added to the LIO-SAM configuration file. Additionally, a transformation between the LiDAR frame and the camera frame must be published for correct calibration. For this purpose, a LiDAR-camera fusion package was used to calculate the intrinsic calibration of the camera using a chessboard, and extrinsic calibration between the LiDAR and the camera, which was based on a manual selection of points in a camera image and a LiDAR point cloud (Krauss, 2025). The camera's intrinsic

calibration values are also automatically provided by the RealSense ROS wrapper. This enables the camera-LiDAR SLAM application to be launched in the same way as the standard LIO-SAM algorithm. However, the application only works if the transformation values between the LiDAR and the camera are correct. Since vibrations can affect the camera's position, calibration should be performed before each use.

## ***2D Mapping and Navigation***

The SLAM Toolbox (Macenski & Jambrecic, 2021) is used to create a 2D map of the environment based on LiDAR data and robot or SLAM-derived odometry. For map generation, SLAM Toolbox expects a message of the type *LaserScan*, which represents a single scan collected from a 2D LiDAR (Laser Range Finder). Since the Hesai XT32 and the Unitree L1 are 3D LiDARs, their point cloud data must be converted into a 2D scan. This can be achieved using a customized version of the *pointcloud\_to\_laserscan* package (Bovbel, 2024; Okolo, 2025), which performs the conversion from 3D point cloud data to a 2D laser scan. Parameters that need to be adjusted include the input point cloud topic, the target coordinate frame for the scan, the vertical range of points to include in the 2D scan, the angular range and resolution of the scan, and valid distances to obstacles. In addition, when integrating with LIO-SAM, the coordinate transformation of the Hesai XT32 LiDAR must be explicitly considered and configured within the source code.

The SLAM Toolbox is integrated directly into ROS2 after installation. Using the 2D laser scan and odometry data, the SLAM Toolbox can continuously generate a map of the environment. This process can be performed offline using recorded data or, as in this thesis,

online in real time. The most important parameters that need to be adjusted in the configuration are the laser range for map rasterizing and the map update interval (Macenski, 2025c). Once map creation is complete, the resulting map can be saved either via a command-line instruction or through the RViz plugin.

Based on the real-time map creation, Nav2, the standard navigation framework for ROS2, is used for robot navigation (Macenski et al., 2020). However, using Nav2, the robot can also be localized and navigated using pre-existing maps. Nav2 can also be integrated directly into ROS2 by installation. Nav2 offers a wide range of navigation plugins, including controller and planner servers for path planning and trajectory control, as well as local and global costmaps that represent the difficulty of traversing different areas. Parameters that should be adjusted depending on the robot and LiDAR sensor include the planning and costmap update frequency, the robot's velocity, the robot's footprint (length and width), the goal tolerance, and the inflation radius around obstacles, which marks areas as difficult to traverse (Macenski, 2025a). After launching the Nav2 application, navigation goals can be set through the RViz plugin. The robot dog will then autonomously move to these destinations, ideally avoiding obstacles along the way.

### **Sensor Configuration and Test Scenarios**

The implemented functionalities are used to assess 3D reconstruction with SLAM, generate high-resolution point cloud scans, as well as mapping and autonomous navigation in indoor environments. For this purpose, the hardware platform needs to be configured.

For the RGB-D camera, the resolution can be set directly when starting the ROS2 node by specifying the color profile. The settings of the Hesai LiDAR can be configured through its

web interface. In particular, the motor’s spin rate (in RPM), which is calculated as the frame rate (in Hz) multiplied by 60, and the return mode must be configured. In addition, Unitree recommends configuring the expansion dock when using SLAM with the Go2 (Unitree Robotics, 2025j). See APPENDIX F: CONFIGURE EXPANSION DOCK FOR SLAM for details. The sensor configuration and operating parameters used in the tests are listed in Table 1.

*Table 1: Sensor configuration and operating parameters*

| Component              | Operating Parameters                               |
|------------------------|----------------------------------------------------|
| Hesai XT32 LiDAR       | Frame Rate: 10 Hz; Return Mode: Last and Strongest |
| Unitree L1 LiDAR       | Frame Rate: 15 Hz                                  |
| RealSense Camera D435i | Resolution: 640×480 @ 30 FPS                       |
| Robot IMU and Odometry | Frequency: 500 Hz                                  |
| Expansion Dock         | Power Mode: MAX                                    |

The robot dog with the developed application was evaluated in a faculty building at the University of Central Florida. The tests aimed to traverse a hallway while scanning it with the Hesai XT32 LiDAR. Three areas are particularly important in this context:

First, the capability of 3D reconstruction using SLAM algorithms and the generation of high-resolution point clouds is assessed. This includes evaluating the quality of the real-time generated 3D point cloud, the positional drift of the robot dog during mapping, and the quality of the final map output (saved as a PCD file). Additionally, the different 3D SLAM approaches are

compared, and the performance is evaluated both when the application runs on the Go2 itself and when it runs on a connected user PC.

Second, the quality and accuracy of the 2D map generated by the SLAM Toolbox are then evaluated. The results are compared using two separate test runs: one with the Hesai XT32 LiDAR and one with the Unitree L1 LiDAR as a mapping sensor. It is also important to determine whether generating an additional 2D map during runtime affects the quality of the 3D mapping.

Lastly, the evaluation investigates the robot dog's ability to autonomously navigate using the Nav2 framework based on the SLAM Toolbox-generated map. Additionally, it will be examined to what extent 3D reconstruction and autonomous navigation can be performed wirelessly in real time with the robot dog.

To ensure a reliable launch of the applications, the robot must first be allowed to stand up and then be unlocked. Subsequently, Go2's General Mode (also referred to as Normal Mode) for motion control should be activated. Finally, Go2's obstacle avoidance system should be deactivated. This entire sequence, performed via the remote controller or mobile app, is mandatory to ensure proper operation and accurate scanning results.

Figure 8 shows the user interface of the launched application in RViz, featuring an overview of the visualized data, including the map with the robot dog and generated point cloud, the SLAM Toolbox plugin for map management, a camera view from the D435i camera, and a control panel for moving the robot dog. A similar interface is used for the navigation application, which visualizes cost maps and planned navigation paths.



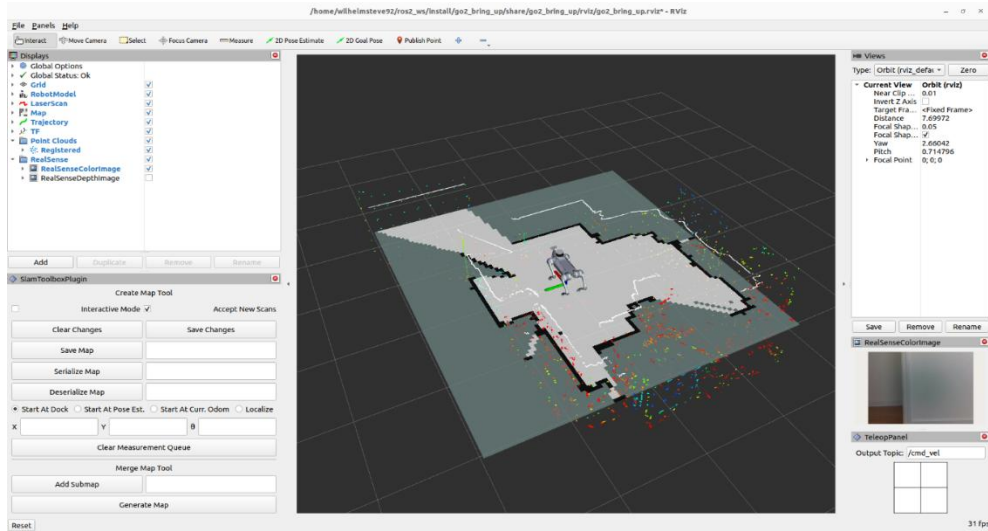


Figure 8: Launched user interface in RViz showing the map, robot dog, point cloud, SLAM Toolbox, camera view, and control panel

## **CHAPTER FOUR: RESULTS**

This chapter presents the results from test scenarios conducted with the Go2 robot equipped with an additional LiDAR sensor and camera to perform simultaneous 3D reconstruction, generate high-resolution point cloud scans, conduct 2D mapping, and execute autonomous navigation, primarily during scans of university building hallways. The investigation covers initial observations of system performance, real-time 3D reconstruction and point cloud density, as well as 2D mapping and the robot's autonomous navigation capabilities. Additionally, different approaches to wireless operation and their reliability in practical scenarios are analyzed. The chapter concludes by summarizing the main contributions of this work

### **Initial Observations**

During the initial test runs, the applications were only executed on the Go2 robot itself. The SLAM algorithms ran smoothly when the Jetson module operated in maximum power mode, provided the robot dog was only moved over a short distance. However, as the size of the 3D scan and the computational needs of the SLAM approaches increased, the application began to stall, sometimes resulting in data loss, particularly affecting the data from the Hesai XT32 LiDAR, interrupting SLAM calculations, leading to position drift or incorrect 3D reconstruction, or termination of real-time live visualization. Additionally, within a few minutes of starting the SLAM application at full power, the expansion dock issued a hot surface warning, indicating a rapid temperature rise. To counteract or delay this behavior, the power mode of the expansion dock can be reduced. However, doing so may lead to insufficient computational resources for running the SLAM application continuously. DLIO occasionally stopped producing reliable

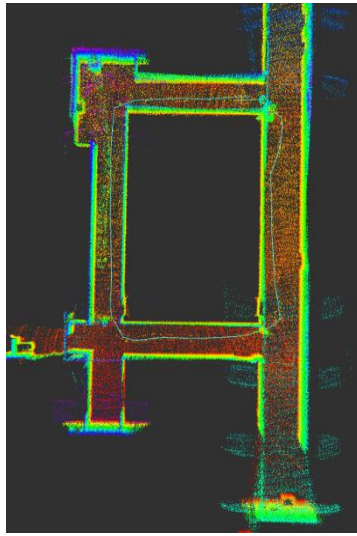
outputs within the first minute of visualization. Due to these performance issues described, subsequent tests were conducted with the applications running on a connected PC.

### **Real-Time 3D Reconstruction and Point Cloud Quality**

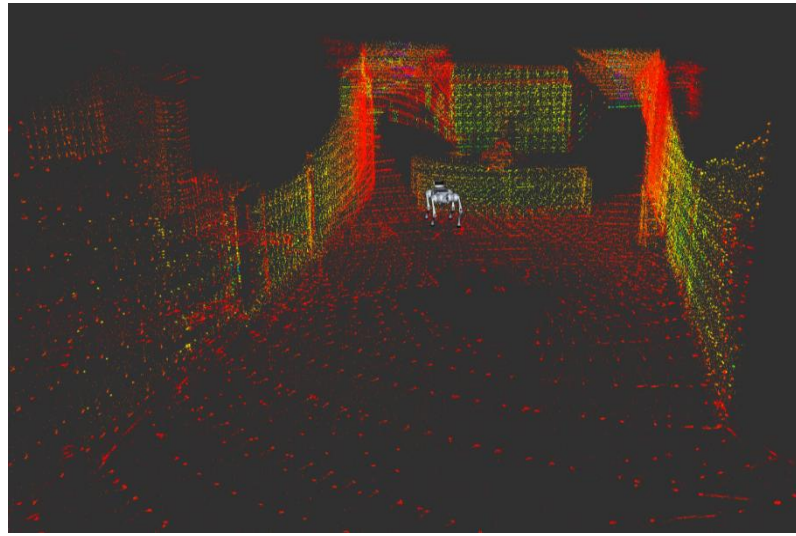
In general, the SLAM approaches were stable and performed well when the robot moved at a moderate pace and avoided sharp turns. In approximately 10% of the test runs, a distorted 3D point cloud appeared in the visualization software immediately after application startup, requiring a restart of the application. Both SLAM approaches can provide two types of 3D point clouds suitable for 3D reconstruction: one that is refined and filtered, and another “raw” point cloud with higher resolution but without filtering. LIO-SAM provides both point cloud types simultaneously, whereas DLIO requires specifying the desired type in the configuration file.

Figure 9 shows a real-time, filtered 3D scan of a hallway segment generated by LIO-SAM using Hesai XT32 LiDAR, presented in RViz from both a high-angle perspective and a rear view of the robot as it moves forward. It should be noted that due to the lack of full IMU pre-integration, the generated point cloud was misaligned, resulting in the vertical center of the point cloud at the height of the Go2’s footprint. Figure 10 shows the “raw” de-skewed 3D scan of the hallway segment. While the resolution and point cloud density are higher in the “raw” map, its visualization requires more computing power, which also led to more frequent shifts in the live visualization.

The results obtained using LIO-SAM demonstrated that surfaces, objects, walls, and corners were accurately captured. Even the ceiling structures were captured from a distance.

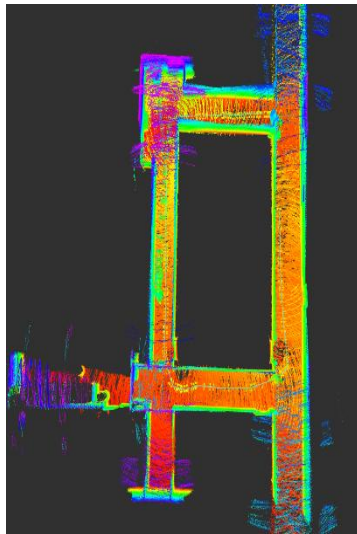


(a) Top-down view

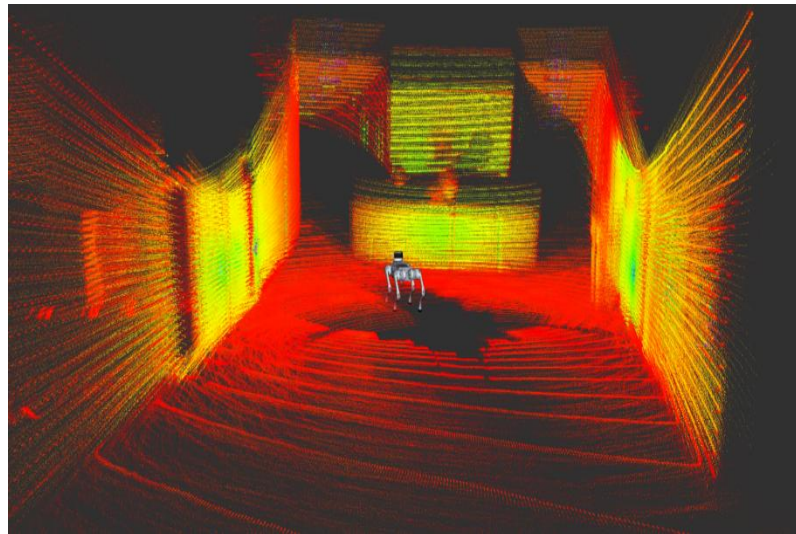


(b) Live view, shows Go2 moving forward in front of a sofa

Figure 9: Real-time 3D scan of a hallway based on the filtered registered point cloud generated by LIO-SAM



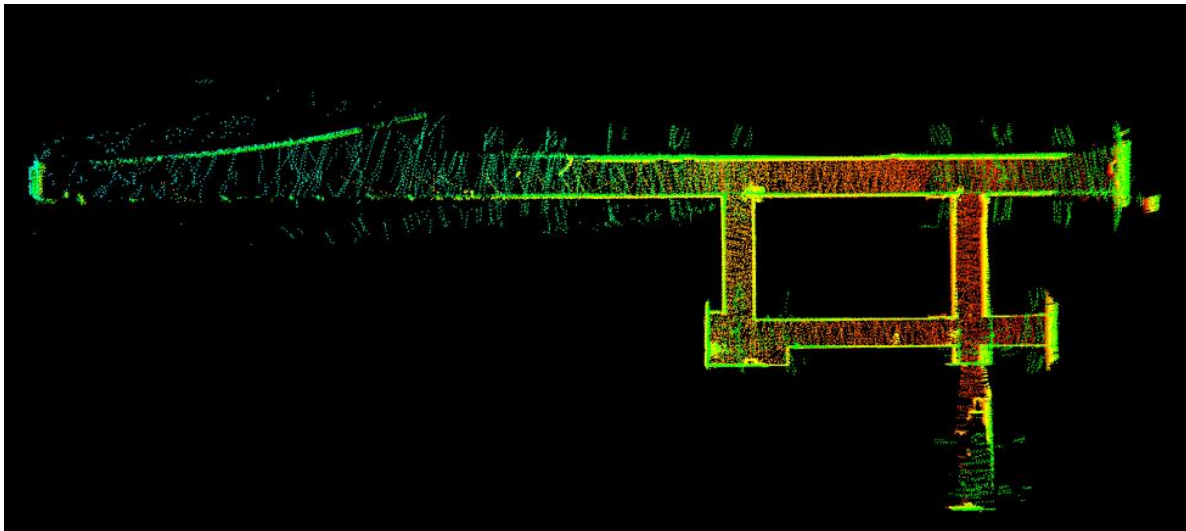
(a) Top-down view



(b) Live view, shows Go2 moving forward in front of a sofa

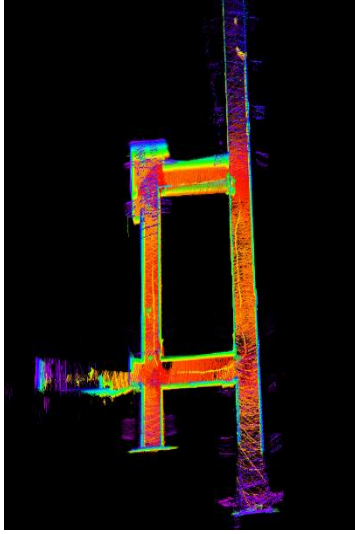
Figure 10: Real-time 3D scan of a hallway based on the “raw” registered point cloud generated by LIO-SAM

However, when using LIO-SAM, a slight drift in the robot’s position during live visualization was observed in approximately 30% of the test runs, resulting in a shifted 3D point cloud. Despite slight deviations in the real-time point cloud live visualization, the final 3D maps, while not the highest in resolution, were accurate and largely free from displacement, as demonstrated by the map displayed in the *pcl\_viewer* in Figure 11.

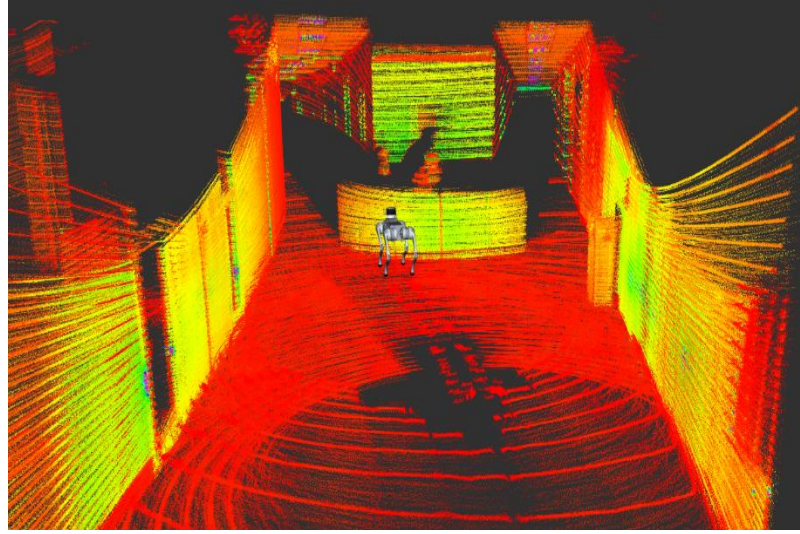


*Figure 11: Bird’s-eye view of the saved 3D map generated by LIO-SAM, visualized using *pcl\_viewer**

The DLIO approach publishes a de-skewed point cloud (“raw” or filtered) corrected for motion distortion, making it suitable for 3D reconstruction. The unfiltered point cloud is also of high resolution, as shown in Figure 12.



(a) Top-down view



(b) Live view, shows Go2 moving forward in front of a sofa

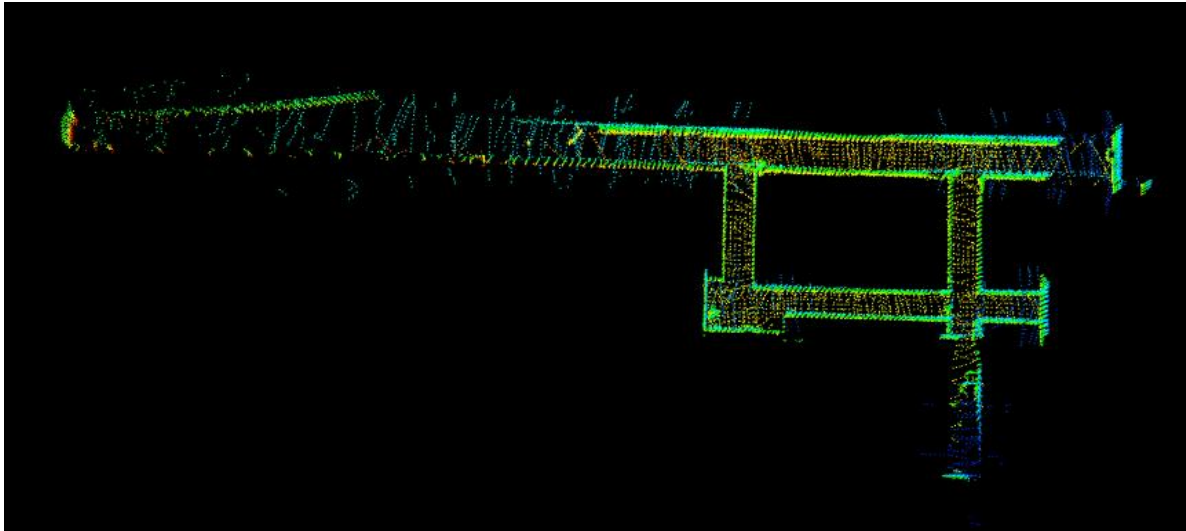
Figure 12: Real-time 3D scan of a hallway based on the “raw” de-skewed point cloud generated by DLIO

Compared to LIO-SAM, DLIO exhibited fewer point cloud shifts on average during real-time visualization. However, it aborted calculations more frequently in narrow spaces and corridors. As with LIO-SAM, slight deviations observed during real-time live visualization of the point cloud with DLIO did not appear in the final saved map, as demonstrated by the map displayed in the *pcl\_viewer* in Figure 13. Furthermore, it should be noted that the final map generated by LIO-SAM has higher resolution and point cloud density than that produced by DLIO. The results obtained using DLIO also demonstrated that indoor environments were accurately captured while the robot dog was in motion.

For longer scans, the computational requirements increased substantially, especially during live visualization of the high-resolution point clouds from both SLAM approaches. This led to slowdowns in the display or even calculation errors. In addition, during scans of larger

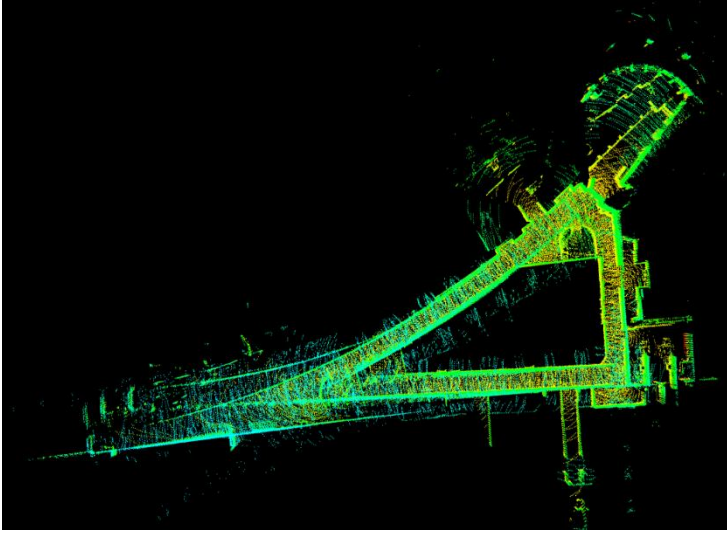


areas, displacements in the 3D point cloud generated by LIO-SAM occurred more frequently than with DLIO, as shown in the map in Figure 14, which displays a scan of a part of the hallway using LIO-SAM. More errors in the generated point cloud also occurred in both approaches when the robot moved too quickly, did not walk stably, or performed overly aggressive movements or turns.

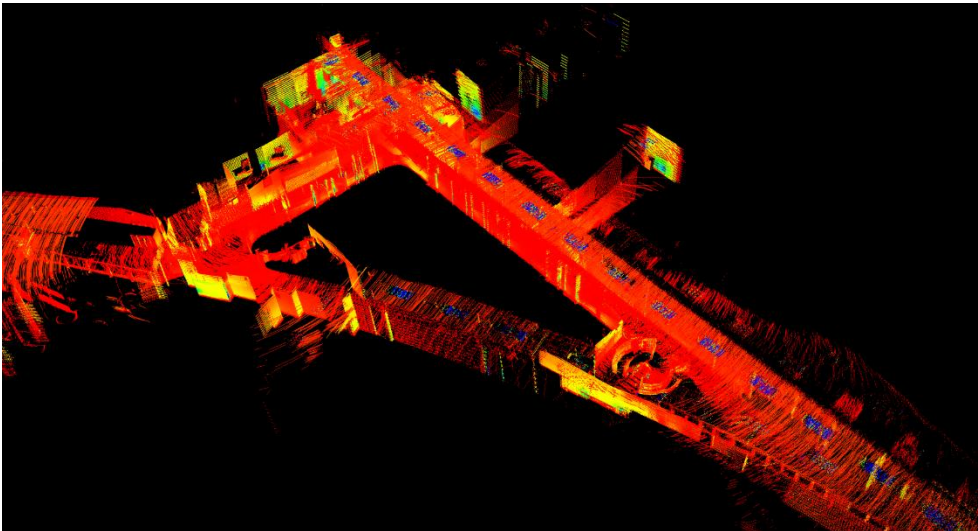


*Figure 13: Bird's-eye view of the saved 3D map generated by DLIO, visualized using `pcl_viewer`*

The *point\_cloud\_accumulator* node enabled the saving of high-resolution point clouds as 3D scans during real-time execution of LIO-SAM and DLIO. However, accumulating the point clouds occasionally stalled, resulting in the loss of some data points. Despite this, the overall quality of the generated point cloud map was not adversely affected, as shown in Figure 15. It is also worth noting that after performing several shorter scanning sessions over approximately 90 minutes, the robot dog's battery level remained at around 50%.



*Figure 14: Larger map generated by LIO-SAM, showing noticeable drifts in the point cloud on the left side*

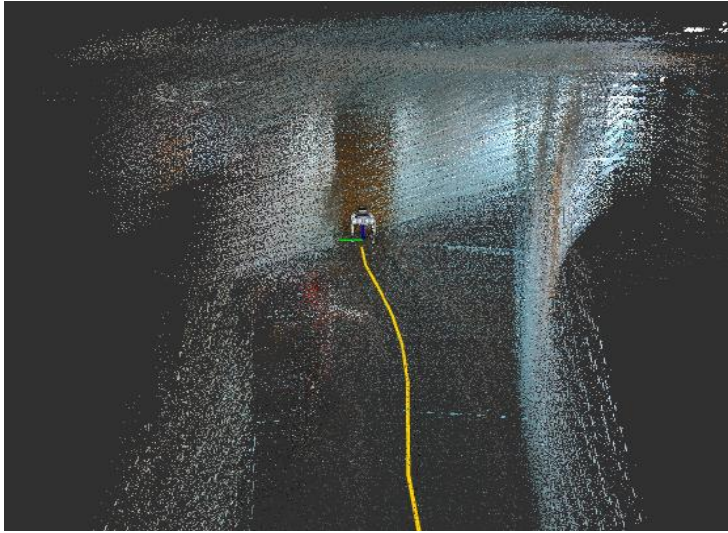


*Figure 15: High-angle view of the saved high-resolution point cloud scan*

In addition to 3D point cloud visualization with LIO-SAM and DLIO, I also assessed how the camera can be used to colorize the map generated by LIO-SAM. The sensor fusion of



camera and LiDAR and the effort to match point coordinates result in a colored point cloud that covers only the camera's field of view, rather than a full 360° point cloud. As a result, scanning environments took longer when aiming to obtain and save a colorized 3D scan. In addition, lighting conditions also greatly affected the colorization of the point cloud. The point cloud presented in Figure 16 is only sparsely or unclearly colorized, mainly due to insufficient calibration. However, it demonstrated that a colorized point cloud of the hallway could be generated using the robot dog and the equipped sensors.

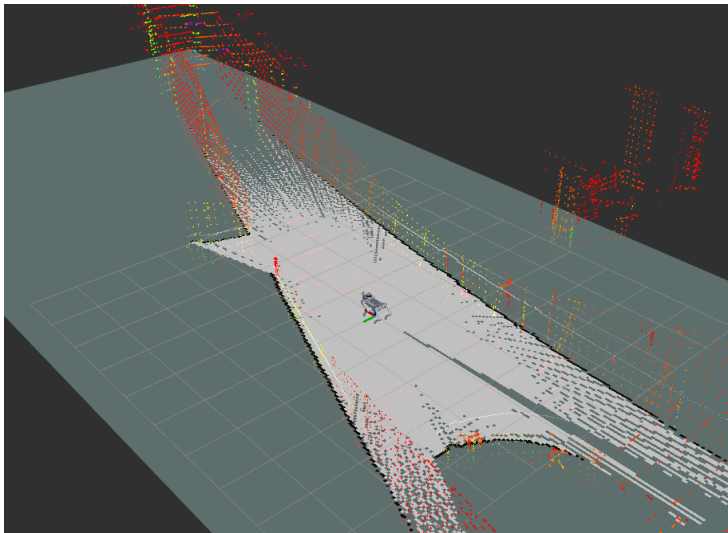


*Figure 16: Real-time, colorized 3D scan showing the Go2 moving forward in the hallway*

All raw sensor data required for SLAM approaches can also be recorded in a *rosvbag2* file and later processed offline by the SLAM algorithms. Offline processing has been observed to generally produce more reliable visualizations with fewer shifts and drifts. However, the generated 3D maps still varied slightly between runs, even when using the same data.

### Simultaneous Mapping and Autonomous Navigation

For 2D mapping, the 3D point cloud from the LiDAR is converted into a 2D laser scan. This scan serves as input to the SLAM Toolbox for map generation, while the odometry data from the 3D SLAM aids the SLAM Toolbox in accurately localizing the robot on the map. Figure 17 shows the application startup with LIO-SAM and SLAM Toolbox using the Hesai XT32 LiDAR. Shown are both the filtered 3D point cloud and the 2D map of the hallway in the university building, generated from the Hesai XT32 point cloud data. Due to the position and range of the XT32, a large and clear map was generated, making it well-suited for autonomous navigation by the robot dog. However, a limitation was that low-lying objects in the immediate vicinity of the robot were not always detected and therefore were not represented on the map.

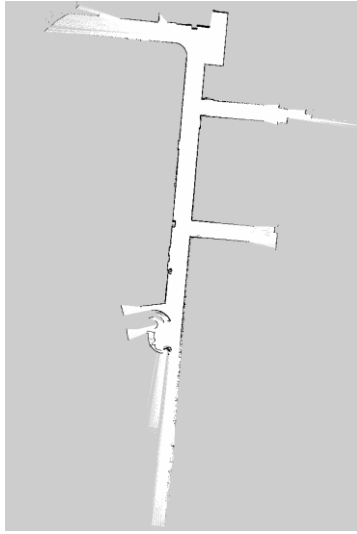


*Figure 17: Real-time generated 2D map of a hallway with the surrounding 3D point cloud*

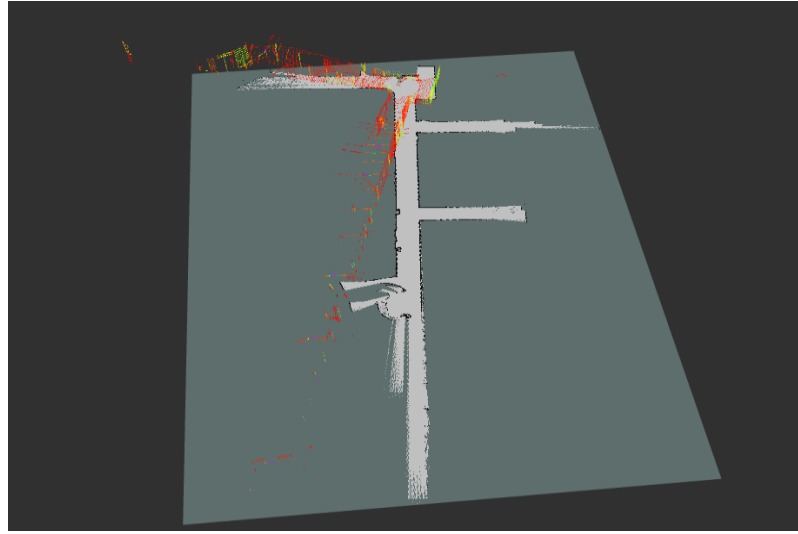
Since SLAM Toolbox further increased the computational load of the application, especially during live visualization, combining 2D and 3D SLAM caused visualization shifts, particularly during longer scans. Figure 18 shows a map generated and saved by SLAM Toolbox at the end of the hallway scan, alongside the live visualization where a noticeable deviation between the 3D scan and 2D map is visible.

In addition to the Hesai XT32, I also used the Unitree L1 LiDAR for 2D mapping, which is mounted underneath the robot dog's head. Due to the operational characteristics and position of this LiDAR, the SLAM Toolbox could only generate a small map with insufficient coverage at a time, as shown in Figure 19. This meant that when using the L1 LiDAR, I needed to move more, for longer periods, and at a slower speed to create an adequate map. This also made it considerably more difficult to define suitable goals for path planning during autonomous navigation. Furthermore, it was observed that using both LiDARs together often resulted in more discrepancies in the visualization of the point cloud and map than when only the XT32 was used for both 2D and 3D mapping.

2D mapping is a prerequisite for deploying the Nav2 navigation framework. In my experiments, I did not rely on pre-built maps for navigation. Instead, the robot dog simultaneously performed 2D mapping and autonomous navigation in real time. To ensure and evaluate meaningful autonomous navigation, the XT32 LiDAR was used primarily as a sensor instead of the L1 LiDAR. Based on the map generated by SLAM Toolbox, Nav2 generated local and global costmaps in real time – 2D grids that represent obstacle costs to help the robot plan safe paths and avoid collisions.



*(a) Saved map generated by SLAM Toolbox*



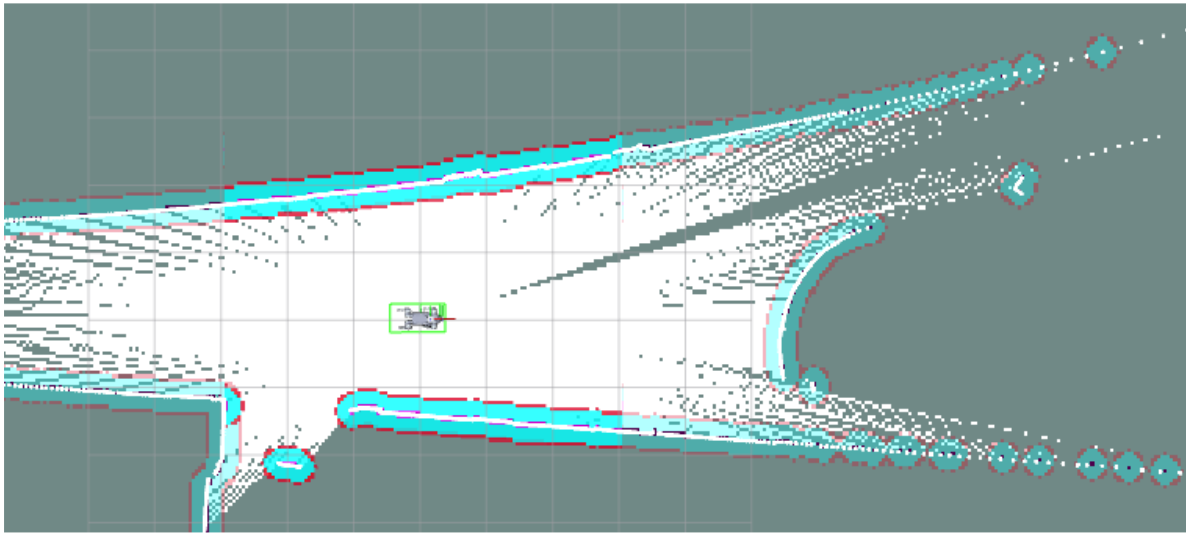
*(b) Live visualization of the 2D map alongside the shifted 3D point cloud from LIO-SAM*

*Figure 18: Generated 2D map of a hallway alongside the 3D point cloud using the XT32 LiDAR*



*Figure 19: Generated map of hallway segment using the L1 LiDAR*

Figure 20 shows the navigation map with the robot dog in the center, displayed in RViz. Darker areas on the map represent obstacles or boundaries that were avoided during path planning and navigation.



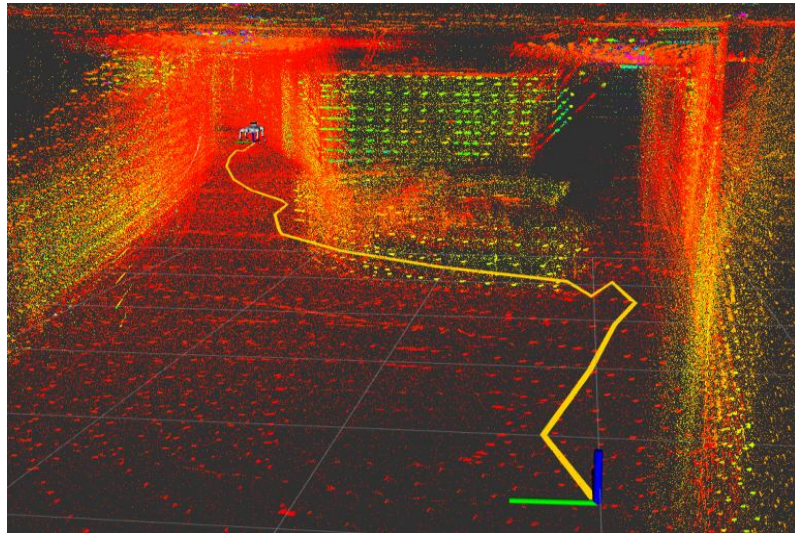
*Figure 20: Navigation map; darker areas represent obstacles avoided during path planning and navigation*

In my test runs, with its Nav2 plugin, RViz was used to set goal points on the map, enabling the robot to navigate there autonomously. Alternatively, RViz could have been used to receive coordinates and set navigation goals via terminal commands. During the runs in the spacious hallway, the XT32's range enabled setting extensive navigation goals that the Go2 reached 100% successfully, avoiding all obstacles and without any collisions. Figure 21 shows the Go2's real-time navigation path – both the straight path between the starting point and the

destination (straight line) and the actual path through the hallway with an intermediate destination to the right of the sofa, demonstrating that the obstacle was avoided.



(a) Navigation path (straight line)

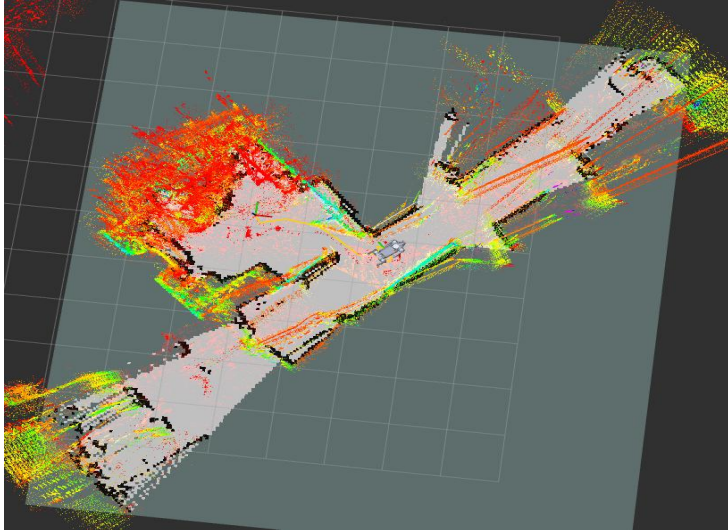


(b) Actual navigation path alongside 3D reconstruction, with an intermediate goal set next to a sofa

*Figure 21: Comparison of the straight-line path between start and goal and the actual navigation path of the Go2, including an intermediate goal near the sofa*

During further tests in tighter environments, such as narrow doorways, it was observed that the Go2 sometimes either collided with obstacles or walls or stopped because no safe path to the target could be found. Stopping behavior could be partially reduced by decreasing the safety buffer zone around obstacles (inflation radius), although this increased the risk of collisions. Especially in tight environments, it was observed that the robot dog made sharp turns or rotations during autonomous navigation, which negatively impacted the generation of the 3D point cloud

and 2D map. Particularly in the live visualization, significant shifts in the point cloud and map, as well as drifts in robot localization, were observed, as shown in Figure 22.



*Figure 22: Generated 2D and 3D maps showing shifts after sharp turns during autonomous navigation in a tight environment*

## **Wireless Operation**

Various approaches to wireless or remote control were assessed. A crucial factor in these tests was maintaining a reliable connection between the user PC and the Go2, either through an available Wi-Fi network or via the network provided by Go2's USB Wi-Fi adapter (hotspot). Running the entire application, including visualization, solely on the Go2 with remote login via remote software was, as described in the initial observations, hardly feasible due to limited computing capacity. A good workaround was to run the computational algorithms on the Go2 while performing the visualization on an external PC. ROS2 natively supports transferring topics

over Wi-Fi using Cyclone DDS. With sufficient Wi-Fi bandwidth, this provided a solid solution for offloading the computationally intensive visualization tasks.

However, during longer scans, the Go2's computing capacity again reached its limits. This, combined with insufficient bandwidth for transmitting many topics, increasingly led to delays in visualization or packet loss during data transmission. This was primarily due to the size of the LiDAR (and camera) data used as input for SLAM. For this reason, it was most practical to run the application and its computations on an external PC, while transferring only essential data, such as IMU, odometry, LiDAR, and camera data, as well as, if necessary, joint motor states for visualizing the robot model, via Wi-Fi from the robot to the external PC. With this setup and sufficient wireless bandwidth, the tests showed hardly any data loss or delays in visualization or in sending navigation commands. However, as soon as the bandwidth was no longer sufficient, severe data loss and interruptions in algorithm computations and visualization occurred, primarily due to packet loss in the LiDAR data. For this reason, it is recommended to compress the LiDAR and camera data on the Go2 using the described compression packages, transmit the compressed data over Wi-Fi to the PC, and decompress it there to run the SLAM algorithms with this data. The results for 3D reconstruction and navigation using lossless *zlib* compression (Adler, 2025) for the LiDAR point cloud were satisfactory compared to a direct PC-to-Go2 Ethernet connection, and remained reliable even with moderate Wi-Fi bandwidth. However, when no Wi-Fi network was available and the connection relied solely on the Go2's USB Wi-Fi hotspot to the PC, limitations arose, primarily due to the range and transmission quality of the Wi-Fi adapter. This was also highly dependent on the environment to be captured, and often quickly reached its limits in winding hallways.



## **Main Contributions**

Based on the development process and the presented results, several key contributions of this work can be identified. The work goes beyond the mere integration of existing tools by actively extending and adapting open-source software components, optimizing system performance, and demonstrating the practical feasibility of simultaneous 3D reconstruction and autonomous navigation using a quadruped robot in indoor environments.

A central innovation is the use of the high-resolution Hesai XT32 LiDAR with the Unitree Go2 EDU robot, an upgrade from the standard XT16 or the integrated Unitree 4D LiDAR commonly used in previous open-source Go2-based SLAM projects (e.g., Serrano, 2025; J. Zhang, 2025). While the Unitree Go2 EDU features a built-in SLAM application, it does not support external LiDAR sensors such as the Hesai XT32 and is therefore only compatible with the Hesai XT16 and Mid-360 (Unitree Robotics, 2025j). To address this issue, a custom ROS2-based system was developed that integrates and extends the two state-of-the-art LiDAR-inertial SLAM approaches, LIO-SAM and DLIO. Both were adapted to support the characteristics of the X32, whose higher resolution and wider scan coverage significantly improve the quality of 3D reconstruction, enabling high-resolution, real-time mapping and navigation.

A key contribution of this work is the integration of real-time 3D reconstruction, 2D mapping, and autonomous navigation into a unified and modular ROS2-based pipeline running concurrently on a quadruped robot. In contrast to many existing approaches that rely on pre-built maps or perform 3D reconstructions offline, this system demonstrates that it is possible to perform all three tasks simultaneously under real-time conditions on a terrain-adaptive mobile

platform. The system architecture enables the flexible exchange of SLAM approaches and sensor configurations, making it adaptable to different use cases and hardware configurations.

Another important improvement is the fusion of multimodal sensor data for enriched 3D perception. LIO-SAM has been specifically extended to support point cloud colorization. Depth data from the Hesai XT32 LiDAR was matched with RGB images from the Intel RealSense D435i camera. This modification goes beyond the capabilities of the native LIO-SAM implementation, which does not support camera integration. While colorization is still dependent on accurate calibration and lighting conditions, it improves the visual quality of the generated point clouds and enables more intuitive scene interpretation.

To enable detailed 3D scans beyond the filtered outputs provided by both SLAM approaches, this thesis introduces a custom *point\_cloud\_accumulator* node that collects and stores raw, high-resolution point clouds in real time. This makes it possible to preserve full-resolution spatial data for offline analysis, further processing, or integration into subsequent tasks.

Finally, the system architecture was evaluated under various wireless operation setups. The results highlight the practical challenges of real-time SLAM over wireless networks and offer a working solution based on selective topic transmission, sensor-side compression, and external compute offloading. These insights are particularly relevant for deployment in environments and scenarios where tethered operation, or constant network availability cannot be guaranteed. To handle the high bandwidth demands of LiDAR and image data transmission, the application leverages existing open-source compression and decompression tools. While these

tools are publicly available, this thesis demonstrates their practical application within a real-time SLAM pipeline on a quadruped robot operating over low-bandwidth Wi-Fi. This configuration enabled reliable data processing, live visualization, and remote operation, highlighting the effectiveness of such compression strategies in real-world, bandwidth-constrained scenarios.

## **CHAPTER FIVE: DISCUSSION**

This chapter reflects on the key findings, analyzes their broader implications, and places them in the context of the original objectives. It begins with an overview of the results and their potential relevance to real-world applications. It then discusses the limitations of the current approach and provides recommendations for future research and system improvements. Finally, it highlights the practical and technical challenges encountered during implementation and highlights lessons learned using the Unitree Go2 EDU.

### **Findings and Implications**

The Unitree Go2 EDU, equipped with an additional 3D LiDAR and an RGB-D camera, has proven capable of simultaneously performing real-time 3D reconstructions, mapping, and autonomous navigation using SLAM algorithms in indoor spaces. While the core functionality operates reliably, performance varies depending on the specific SLAM implementation and sensor configuration, and the complexity and layout of the environment.

The scan results showed that the high-resolution LiDAR accurately captures indoor environments, even while the robot is in motion. This enables the generation of dense and detailed point clouds, supporting effective real-time 3D reconstruction, even with lightweight algorithms. However, reconstruction inaccuracies and positional drift became more noticeable during longer or more complex scans. Autonomous navigation and 2D mapping, based on basic frameworks such as the SLAM Toolbox and Nav2, were found to be reliable and achieved good path fidelity in simple scenarios. In contrast, performance degraded when maps were generated with lower-quality sensors.

Running 2D and 3D SLAM simultaneously, especially in combination with live visualization, resulted in increased computational load. This introduced the risk of degraded output quality and higher inaccuracy, especially in more complex and longer scenarios, and when no external processing unit was used. Multi-sensor fusion shows promising potential to enhance 3D reconstruction quality by integrating diverse sensor data. However, practical challenges such as sensor noise, obstacles, narrow spaces, lighting conditions, and calibration issues can still affect performance.

Real-time wireless operation worked well within short-range local networks with high bandwidth. Even in low-bandwidth conditions, the application achieved stable results using compression techniques but reached its limits when operating under unstable connections. The integration of open source frameworks, especially established SLAM algorithms and ROS2, proved functionally effective and offers flexible interfaces for a wide range of applications, but requires careful configuration effort and a domain-specific onboarding phase to use them effectively.

Overall, the findings indicate that such a system offers practical benefits for real-world applications such as reconnaissance or inspection tasks, particularly in GPS-denied environments or scenarios with limited visibility, where mobile autonomy and spatial awareness are essential. However, to enable robust deployment in operational settings, further improvements are needed in computational efficiency, mapping accuracy, wireless data transmission, multimodal sensor integration, and adaptability to dynamic conditions. Where possible, compute-intensive tasks, such as visualization of high-resolution data in real time, should be minimized, and shorter scans

should be preferred to reduce computational effort and support more efficient 3D reconstruction and mapping.

Although not explored in this thesis, the idea of combining a quadruped robot with a drone for collaborative mapping and exploration offers great potential. While a drone can effectively scan large outdoor areas and the exterior of buildings, a quadruped robot is well-suited for autonomous indoor navigation and scanning. Provided that a reliable network connection is maintained, this combination could enable comprehensive and coordinated 3D reconstruction and mapping across varied environments.

### **Limitations and Recommendations**

Although the results demonstrate that the quadruped robot successfully performs simultaneous 3D reconstruction and autonomous indoor navigation, this work still has limitations that should be addressed in future work.

First, in this work, two LiDAR-inertial odometry SLAM approaches for 3D point cloud reconstruction using the Go2 EDU were implemented as examples: the widely used LIO-SAM approach and the newer, lightweight DLIO SLAM. Numerous other SLAM approaches differ in the types of sensors used, the underlying algorithms, their susceptibility to error, as well as the quality of the resulting 3D point cloud. While the point cloud density in the generated maps produced by the two SLAM methods used in this work is satisfactory, comparison with the saved raw point cloud data reveals room for improvement, suggesting that alternative tools, point cloud aligning and processing algorithms, or SLAM approaches, such as LidarView (Kitware, 2025), should be explored in future work to achieve full point cloud density and high resolution. To

improve the saving process of high-resolution point clouds, this functionality could be integrated directly into the source code of the SLAM algorithms, or more advanced tools for converting point clouds to PCD files could be utilized (Macenski, 2025b). Additionally, the LIO-SAM implementation in this work lacked full IMU pre-integration, which resulted in a misalignment between the generated point cloud and the actual position of the LiDAR sensor. These limitations should be addressed when using this approach in the future to improve reconstruction accuracy. To enhance the quality of the 3D point cloud, interference regions in the immediate vicinity of the XT32 should be masked out to prevent them from appearing in the 3D reconstruction. This particularly applies to the camera mounted on the Go2's head as well as the rear section of the robot dog. To further improve semantic interpretation, reconstruction quality, and enhance the robot's perception capabilities, future implementations may benefit from the integration of advanced techniques such as deep learning-based point cloud segmentation and object detection.

Second, only standard frameworks and procedures for scanning and navigation were employed, without a specific focus on optimizing the quality of the scanning and navigation processes. As also highlighted by D. Hu et al. (2023) and Gan et al. (2024), factors such as lighting conditions (especially when using visual sensors or cameras), scanning techniques, and the robot's scanning poses strongly influence the quality and accuracy of the resulting 3D point cloud. Furthermore, a comparative evaluation of the efficiency of a coupled or decoupled method for 3D reconstruction, mapping, and navigation, as conducted by Gan et al. (2024, 2025), could not be performed due to the insufficient capability of the L1 LiDAR. This issue should be addressed in future research. Additionally, advanced techniques and algorithms for path planning, navigation, and obstacle detection and avoidance could be integrated to enable

smoother movement and enhance overall system performance. For example, instead of manually setting navigation goals, autonomous exploration and goal selection could be implemented using approaches such as frontier-based exploration (da Silva Lubanco et al., 2020; Yamauchi, 1997). While SLAM approaches also perform offline 3D reconstructions using recorded data, offline mapping using the SLAM Toolbox has not yet been implemented. Appropriate adaptations could be made in future work.

Third, although the Hesai XT32 sensor performed well for 2D mapping as a basis for navigation, the LiDAR's positioning on the Go2 limits its ability to detect close-range objects and boundaries at low altitudes. While this limitation may not have posed an issue in the long hallways of the university building, it could lead to challenges in autonomous navigation within more complex and cluttered environments, which were not extensively tested in this thesis. Furthermore, although the position of the L1 LiDAR is well-suited for capturing the area in front of and beneath the robot, the selection and configuration of the SLAM Toolbox for 2D mapping were inadequate to produce a sufficiently reliable map, let alone support meaningful autonomous navigation. Therefore, alternative methods, such as the Point-LIO algorithm provided by Unitree and tailored for Unitree LiDARs, should be explored to better utilize the L1 LiDAR (Unitree Robotics, 2024d; J. Zhang, 2025). Alternatively, additional 2D or 3D LiDAR sensors, or other types of sensors, could be employed to improve 2D mapping capabilities.

Fourth, sensor fusion capabilities for improving 3D reconstruction and navigation were used only to a limited extent. Specifically, only the RGB stream of the RealSense camera was utilized to colorize the point cloud. Furthermore, only an initial and insufficient calibration was performed, leaving substantial room for improvement in the quality of the generated colored



point cloud. For effective sensor fusion involving cameras, LiDARs, or other sensors, robust calibration, preferably using advanced frameworks or methods, should be performed before each deployment. In future implementations, additional data streams from other cameras could also be integrated. For example, the Go2's built-in front camera could be utilized, or the RealSense camera could be replaced with a 360° camera to provide a more comprehensive view. Moreover, in addition to the RGB stream, the camera's depth stream could also be utilized, either to enrich the point cloud for enhanced 3D reconstruction or to serve as an additional or standalone input for SLAM and navigation. For example, alternative visual SLAM or multi-sensor SLAM systems such as RTAB-Map (Labbé & Michaud, 2014, 2019) or R3LIVE (J. Lin & Zhang, 2022), as well as the Go2's internal odometer services interface (Unitree Robotics, 2024c), could be employed to improve 3D scanning, localization, and mapping performance.

Fifth, the robot dog itself presents certain limitations. One significant constraint is the limited hardware resources of the Go2 platform. The onboard Jetson module does not provide sufficient performance to reliably run SLAM or other computationally intensive applications over extended periods, nor is it capable of visualizing large volumes of data in real time. To address this, it is recommended to run such applications on an external PC. To avoid data loss during wireless transmission and reduce dependence on long Ethernet cables, a compact yet powerful PC could be mounted directly on the docking station and connected to the robot. Remote access to this PC would allow applications to be executed and monitored in real time. For improved visualization performance, Foxglove, a more powerful alternative to RViz, could also be employed (Foxglove Technologies, Inc., 2025). Furthermore, for wireless operation and application execution, the Go2 relies on a high-performance Wi-Fi adapter and a stable network

connection that supports Cyclone DDS multicast communication. However, such network conditions may not always be available. For instance, during testing in the university hallways, only the hotspot provided by the USB Wi-Fi adapter could be used as a network connection. This setup limited both the range and performance of the robot dog's applications, even with the implementation of data compression techniques. To improve stability when navigating complex terrain or overcoming obstacles, future projects could consider using the Unitree Go2-W, a further development of the Go2 that features wheels on its legs (Unitree Robotics, 2025m). For applications requiring even greater performance, payload capacity, and robustness, the Unitree B1, B2 or B2-W platforms offer enhanced capabilities suitable for more complex and demanding environments (Unitree Robotics, 2025g, 2025h, 2025l).

Finally, this thesis was limited to the use of a single quadruped robot for 3D reconstruction and mapping tasks. To improve mapping accuracy and scanning quality, multiple robots could be deployed simultaneously to perform these tasks even more efficiently and reduce the computational workload for a single unit. Moreover, extending the system with an aerial component, such as an unmanned aerial vehicle, could further enhance scanning and exploration capabilities, particularly in complex or multi-level environments. However, coordination and communication between the platforms may introduce additional technical challenges and should be investigated in future work.

### **Challenges During Implementation**

Several challenges arose during the application development process that affected progress and caused delays. These challenges, which involved both hardware components (i.e.,

the Unitree Go2 robot) and software aspects (i.e., development using ROS), should be considered for future work.

The Unitree Go2 is a relatively new quadruped robot, and its firmware and software are still under active development. During this thesis, two hardware issues, both acknowledged by Unitree, occurred frequently and had a notable impact on the development process. One of these issues involved the L1 LiDAR, a low-cost sensor developed by Unitree. Throughout development, frequent scan interruptions and inconsistent LiDAR data publication were observed. As a result, several scans with the L1 had to be aborted. Furthermore, a malfunction rendered the L1 LiDAR completely inoperable. Obtaining a replacement unit took several weeks, during which LiDAR-based tasks with the integrated LiDAR could not be performed. Meanwhile, Unitree has released the L2 LiDAR, an improved version of the L1, that offers enhanced technical specifications (Unitree Robotics, 2025k). Given its compatibility with the Unitree Go2, the L2 LiDAR presents a suitable and potentially more reliable alternative for future applications. A second recurring issue involves the Go2's body motor. When the robot remains stationary, the motor may overheat, which can result in the robot crashing and its status indicator flashing red. This issue, attributed to winding overheating of the body motor, led to several system failures during scanning sessions when pauses were required, forcing the cancellation of scans. Despite multiple firmware updates, a permanent solution to this problem has not yet been identified by Unitree.

During testing of the Go2's wireless control capabilities, significant challenges were observed in both the network configuration and Cyclone DDS communication setup. First, setting up a hotspot using the USB Wi-Fi adapter typically only worked if no other Wi-Fi

connections were saved on the system. Second, configuring unicast communication via Cyclone DDS proved incompatible with the Go2, resulting in repeated failures. In both cases, the system experienced network or system crashes that could only be resolved by clearing the network configuration and restarting the robot. Furthermore, the transmission of internal topics from the Go2 over Wi-Fi was inconsistent. This issue impacted key functions such as the visualization of the robot model in RViz and the execution of autonomous navigation commands, thereby preventing comprehensive testing of the wireless control capability. The partial failure of Wi-Fi-based data transmission could only be resolved by reloading a factory image onto the expansion dock. This malfunction was particularly noticeable following network configuration changes, such as switching to a hotspot or modifying Cyclone DDS settings. Although these patterns were observed repeatedly, the exact cause of the issue remains unidentified. Since it is possible to reload the factory image onto the expansion dock but not to perform a full factory reset on the Go2 itself, future research should consider testing additional Unitree Go2 units to rule out device-specific hardware faults.

Another challenge emerged after the installation of new firmware updates via the Unitree app. The Go2 supported several modes for motion control, including General Mode/Normal Mode (default), which offered stable locomotion and access to a wide range of operational commands. In addition, Advanced Mode and AI Mode were available to adapt the robot's movement to more complex terrain, varying surface conditions, and the traversal of obstacles (Unitree Robotics, 2025d). While General Mode yielded the most stable results for scanning tasks, AI Mode proved less suitable due to certain unpredictable, reflexive movements in response to surface conditions or nearby obstacles, such as sudden bending or foot placement

against walls. These adaptive behaviors occasionally introduced shifts in the generated 3D point cloud, reducing the consistency and reliability of the scanning output. While the robot initially started in General Mode and allowed for easy manual switching between modes, a recent update replaced this startup behavior with the so-called Agile Gait Mode. This mode enables an intelligent transition between Normal Mode and AI Mode. Consequently, achieving stable scan results now requires manually switching the mode – an action that remains possible via the mobile app or remote controller but is no longer immediately apparent. Additionally, the Unitree SDK has also been updated to reflect these changes (Unitree Robotics, 2025f). With ongoing firmware updates, the application must therefore be regularly checked for compatibility and adjusted if necessary.

Another major challenge was the implementation with ROS. Although the robot itself and its sensors are ROS-compatible, which generally facilitates integration, and suitable ROS repositories for many SLAM and navigation frameworks exist, using ROS presents some difficulties, particularly regarding the different ROS distributions. A key issue is the limited backward compatibility, especially between ROS1 and ROS2. Newer features, such as those supported by ROS2 Humble, are not available in older versions, such as ROS2 Foxy, which is pre-installed on the Go2 EDU. To still be able to use modern features, the use of Docker containers on the Go2 is essential. Another issue is that some SLAM algorithms have so far only been implemented for ROS1, and no official ROS2 version exists. Therefore, before starting any development work, it is critical to check whether the required repositories are available for the target ROS distribution. Otherwise, developers must prepare either to spend significant time adapting code manually or to accept limitations in functionality. The Go2 itself already publishes

many ROS2 topics by default, including topics of the L1 LiDAR data. However, if these data are to be used in ROS1 or accessed through custom Python or C++ applications, they must first be retrieved via the SDK or corresponding software repositories. Furthermore, with the EOL of ROS Noetic (ROS1) and ROS Foxy (ROS2), support and updates will no longer be provided for these distributions. This raises the question of whether future development of ROS-based applications for the Go2 might face constraints or whether Unitree will offer official upgrades. Currently, Unitree does not recommend manually upgrading the ROS version on the docking station to preserve the full functionality of the system.

## CHAPTER SIX: CONCLUSION

The thesis aimed to develop an application that enables the quadruped robot Unitree Go2 EDU, equipped with a Hesai XT32 LiDAR sensor and a RealSense D435i RGB-D camera, to autonomously navigate and simultaneously perform real-time 3D reconstruction of indoor environments. By integrating open-source SLAM algorithms and a navigation framework into a ROS2 application, the platform was able to generate high-resolution point clouds while navigating autonomously in a hallway of a university building, which allowed for controlled evaluation of sensor performance, SLAM accuracy, and navigation stability.

The results show that accurate 3D mapping and navigation in real time are possible, although the quality and reliability of 3D reconstruction and SLAM algorithms depend on several factors. These include the choice of algorithm, the quality and calibration of the sensors, the type and depth of the live visualization, and the computing capacity of the system. While dense and detailed point clouds were successfully generated, longer or more complex scans resulted in increased drift and inaccuracies in reconstruction. Furthermore, simultaneous 2D and 3D SLAM, especially when combined with real-time visualization, imposed a significant computational load that could impact performance on limited hardware, especially when the entire application is running only on the Go2. To ensure stable remote control and data transmission, a high-bandwidth wireless network connection was essential.

Despite these limitations, the application has effectively fulfilled its goal and highlights the potential of a quadruped robot combined with suitable sensors for 3D scanning or even reconnaissance tasks in indoor environments. Overall, this thesis contributes a functional

application for mobile and autonomous 3D scanning using a quadruped robot and open-source software, offering a replicable and extensible platform for future research in autonomous 3D reconstruction and mapping. The use of ROS2 and open-source frameworks provides a flexible foundation but requires careful integration and configuration effort.

Based on the insights gained during development and testing, future work should focus primarily on four key areas. First, improving wireless communication for more robust remote operation. Second, optimizing SLAM performance through multi-sensor fusion approaches such as RTAB-Map (Labbé & Michaud, 2014, 2019). A robust camera LiDAR calibration tool should also be used for this purpose. Furthermore, the D435i camera could be replaced with a 360° camera to cover a larger area for fusion. Third, integrating advanced navigation strategies such as frontier-based exploration (da Silva Lubanco et al., 2020; Yamauchi, 1997) to improve autonomy and adaptability in complex or dynamically changing environments. Finally, expanding the system by enabling collaboration between multiple quadruped robots or aerial drones could further enhance autonomous mapping and reconstruction in complex, real-world scenarios. Recent open-source implementations on the Unitree Go2 quadruped robot have successfully showcased the integration of RTAB-Map, autonomous exploration, and multi-robot SLAM (Bhattacharya, 2025; Naderi, 2025), providing a valuable foundation for future system extensions.



## **APPENDIX A: EXPANSION DOCK RECOVERY AND UPDATE**

Resetting the expansion dock is necessary if errors occur during application programming or when a fresh system state is required. A technically reliable hardware reset is not available directly on the Go2 itself. First, you need to obtain the factory image. You can find it in the Unitree Go2 SDK Development Guide (Unitree Robotics, 2025c). The 100 TOPS Jetson Orin NX factory image is required for the Go2 Edu used in this thesis. To flash the image, a compatible 256GB NVMe storage device with a USB adapter is required. The Go2 Edu used in this thesis is equipped with a Western Digital 256GB M.2 2242 PC SN530 NVMe SSD. The following steps are required to flash the image onto the NVMe device with Ubuntu OS:

1. Insert the NVMe storage device into a Linux host computer.
2. Identify the storage device by running `$ lsblk -f` in the terminal.
3. Unmount the device with `$ sudo umount /dev/sdX` (replace sdX with the actual device node, e.g., sdb1).
4. Format the device to the ext4 filesystem with `$ sudo mkfs -t ext4 /dev/sdX` (replace sdX with the device node).
5. Navigate to the directory containing the factory image (with or without CUDA installed) and flash the image onto the device by running `$ bzip2 -dc nx.img.bz2 | sudo dd of=/dev/sdX status=progress` (replace sdX with the device node).
6. After flashing, safely power off the NVMe device with `$ sudo udiskctl power-off -b /dev/sdX` (replace sdX with the device node).
7. Remove the NVMe storage device and insert it back into the internal NVMe slot of the expansion dock (note: the expansion dock must be disassembled for this step, see Figure 4).

8. Power on the Go2, and the system will boot from the new internal NVMe storage.

For the services installed on the expansion dock, such as the SLAM and navigation services interface or the odometer services interface, Unitree may also provide new updates to improve functionality. Therefore, it is important to check for and install module updates. New module updates can be found in the Unitree Go2 SDK Development Guide (Unitree Robotics, 2025c). The following steps are required to update the expansion dock module:

1. Connect your PC to the RJ45 Ethernet port on the rear of the expansion module and establish an SSH connection by running the command `$ ssh unitree@192.168.123.18`. When prompted, enter the default password `123` to access the docking station.
2. Open a web browser and navigate to `http://192.168.123.18`, where the update interface will load, initially showing no package selected on the left side.
3. Finally, select and upload the downloaded update package from your file system via the web interface, start the update process, and wait until it completes.

## **APPENDIX B: INITIAL EXPANSION DOCK SYSTEM SETUP**

Before starting application development, an initial setup of the expansion dock on the Ubuntu OS should be performed, which includes the installation of essential packages as well as system configurations.

On your Linux PC:

1. Install a ROS2 distribution, such as ROS 2 Humble, and NoMachine.
2. Set the IP address of the PC to be in the same network segment as the Go2, for example: 192.168.123.200.
3. Connect the PC and the Go2 using an Ethernet cable and confirm the connection by running: *\$ ping 192.168.123.18*.

SSH into the Go2 by running the command *\$ ssh unitree@192.168.123.18*, password *123*, and start NoMachine's NX server by running *\$ ./nomachine.sh*. Then, use the NoMachine client from your PC to enable remote login. To establish a network connection, connect a mobile phone to the expansion dock via USB and activate USB tethering. Run *\$ sudo apt update && sudo apt upgrade -y* on the Go2 to update the system, then perform a *\$ sudo reboot*. After the system restarts, start the NX server again to enable remote login. Afterward, a USB Wi-Fi adapter should be connected and the corresponding driver installed to enable a permanent network connection (see APPENDIX C: INSTALL USB WI-FI ADAPTER). Furthermore, the expansion dock should be updated (see APPENDIX A: EXPANSION DOCK RECOVERY AND UPDATE). After that, install additional tools by running the following commands in the Ubuntu terminal:

```
# Install a set of helpful tools and utilities on Go2 by running:

sudo apt install -y nano chrony git pcl-tools meld tmux

pip3 install --user catmux

echo `export PATH="$HOME/.local/bin:$PATH"` >> ~/.bashrc


# For using the RealSense camera, install the necessary packages:

sudo apt install -y ros-foxy-realsense2-camera \

                    ros-foxy-realsense2-description \

                    ros-foxy-image-transport \

                    ros-foxy-image-transport-plugins

# After installation, reboot the system:

sudo reboot


# Check if the camera is properly recognized:

rs-enumerate-devices


# If the RealSense camera is not recognized via the USB-C port, try:

sudo apt install -y linux-image-generic linux-headers-generic


# Set the correct system time zone:

sudo timedatectl set-timezone America/New_York
```

If the Go2 is not connected to the internet, its system clock resets to 1970 upon powering on, which can cause issues with ROS applications. Connecting the expansion dock to Wi-Fi will automatically update the clock; otherwise, you can use your connected PC as a time synchronization server:

On your PC:

```
# Install the chrony time synchronization service

sudo apt install chrony


# Edit the chrony configuration file

sudo nano /etc/chrony/chrony.conf


# Add the following two lines at the bottom of the file

local stratum 10

allow 192.168.0.0/16 # adjust subnet as needed or add more, e.g., 10.0.0.0/8


# Restart the chrony service

sudo systemctl restart chrony
```

## On the Go2:

```
# Edit the chrony configuration file

sudo nano /etc/chrony/chrony.conf

# Add the following line, replacing 192.168.123.222 with your PC's IP:

server 192.168.123.222 iburst

# Restart the chrony service

sudo systemctl restart chrony

# Verify that the PC is listed as a time source

chronyc sources -v
```



## **APPENDIX C: INSTALL USB WI-FI ADAPTER**

For the initial internet connection, connect a smartphone to the Go2 via USB and activate USB tethering. Then, download and install the Wi-Fi adapter driver. In this thesis, a BrosTrend USB Wi-Fi adapter was used. To install its driver, connect the adapter and run `$ sh -c 'wget linux.brostrend.com/install -O /tmp/install && sh /tmp/install'`. After completing the installation, you can verify that the adapter is recognized by running `$ iwconfig`. A Wi-Fi adapter device should appear in the list. To allow *NetworkManager* to manage the adapter, update its configuration by running `$ sudo sed -i 's/managed=false/managed=true/' /etc/NetworkManager/NetworkManager.conf`. Then, restart the *NetworkManager* service with `$ sudo systemctl restart NetworkManager`. To connect Go2 to a Wi-Fi network, use `$ sudo nmcli device wifi connect "<SSID>" password "<pwd>"`.

If the Wi-Fi adapter supports AP (Access Point) mode, you can create a wireless hotspot that your PC can connect to. To create the hotspot, for example, run `$ sudo nmcli dev wifi hotspot ifname wlan0 ssid go2_usb_hotspot password "hotspot0000"`. You can verify the connection with `$ nmcli connection show`. If you need to manually bring a saved connection down or up, such as the hotspot, you can use `$ sudo nmcli connection down Hotspot` or `$ sudo nmcli connection up Hotspot`. To remove a saved connection, such as the hotspot, run `$ sudo nmcli connection delete Hotspot`.

## **APPENDIX D: SETTING UP ROS2 SUPPORT OF UNITREE ROBOTS**

The typical environment for setting up ROS2 support for Unitree robots is the *setup.sh* file is as follows, assuming Go2 is connected via Ethernet (Unitree Robotics, 2025h).

```
# Source ROS2 setup script to set up environment variables.

source /opt/ros/humble/setup.bash

# Source setup script of Cyclone DDS to overlay it on the environment.

source $HOME/unitree_ros2/cyclonedds_ws/install/setup.bash

# Set the ROS Middleware implementation to Cyclone DDS.

export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

# Configure Cyclone DDS to use network interface eth0 for communication.

export CYCLONEDDS_URI='<CycloneDDS><Domain><General><Interfaces>

<NetworkInterface name="eth0" priority="default" multicast="default" />

</Interfaces></General></Domain></CycloneDDS>'
```

## **APPENDIX E: DOCKER INSTALLATION ON GO2 EDU**

To run applications developed with ROS2 Humble, an appropriate Docker container must be set up on the Go2. To install Docker correctly, the following commands must be executed:

```
sudo apt-get update

sudo apt-get install apt-transport-https ca-certificates curl \

software-properties-common

curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \

sudo gpg --dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg

echo "deb [arch=$(dpkg --print-architecture) \

signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] \

https://download.docker.com/linux/ubuntu \

$(lsb_release -cs) stable" | \

sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

sudo apt-get update

sudo apt-get install docker-ce docker-ce-cli containerd.io

sudo usermod -aG docker $USER

newgrp docker
```

```
distribution=$(. /etc/os-release;echo $ID$VERSION_ID) && curl -s -L \
https://nvidia.github.io/nvidia-docker/gpgkey | \
sudo apt-key add - && curl -s -L \
https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list | \
sudo tee /etc/apt/sources.list.d/nvidia-docker.list

sudo apt-get update

sudo apt-get install -y nvidia-docker2

sudo systemctl restart docker
```

If errors occur during *\$ sudo apt-get install -y nvidia-docker2*, run:

```
sudo apt --fix-broken install

sudo apt-get update

sudo apt-get install -y nvidia-docker2

sudo systemctl restart docker
```

After the installation completes successfully, navigate to the directory containing your Docker image and run:

```
docker build -t go2-humble:latest .
```

## **APPENDIX F: CONFIGURE EXPANSION DOCK FOR SLAM**



To run SLAM on the Go2, Unitree recommends configuring the expansion dock to handle the computational needs and buffer size demands of connected sensors and SLAM algorithms (Unitree Robotics, 2025j):

1. Set the power mode of the expansion dock to 0: *MAXN*.
2. To modify the maximum receive buffer size to 50 MB, add the following line to the Go2's *cyclonedds.xml* file, as well as to your *setup.sh* file on your PC and inside the Docker image, if used:

```
<Internal>

    <MinimumSocketReceiveBufferSize>50MB</MinimumSocketReceiveBufferSize>

</Internal>
```

3. Additionally, on both the Go2 and your PC, run the following commands to increase the maximum receive and send buffer sizes:

```
sudo su

echo "net.core.rmem_max=52428800" >> /etc/sysctl.conf

echo "net.core.wmem_max=52428800" >> /etc/sysctl.conf

sysctl -p
```

## LIST OF REFERENCES

- Abdulmajeed, W., & Mansoor, R. (2014). *Visual Robot SLAM of 2D & 3D Indoor Environment*.
- Adler, M. (2025, May 18). *Zlib*. <https://github.com/madler/zlib>
- Ames, A. D., Coogan, S., Egerstedt, M., Notomista, G., Sreenath, K., & Tabuada, P. (2019). Control barrier functions: Theory and applications. *2019 18th European Control Conference (ECC)*, 3420–3431. <https://doi.org/10.23919/ECC.2019.8796030>
- Besl, P. J., & McKay, N. D. (1992). A method for registration of 3-D shapes. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 14(2), 239–256. <https://doi.org/10.1109/34.121791>
- Bhattacharya, S. (2025). *Multi-hetero-agent exploration on Unitree Gos*. <https://github.com/Sayantani-Bhattacharya/Multi-Hetero-Agent-Exploration-on-UnitreeGos>
- Bovbel, P. (2024). *Pointcloud to laserscan*. [https://github.com/ros-perception/pointcloud\\_to\\_laserscan](https://github.com/ros-perception/pointcloud_to_laserscan)
- Chen, K., Nemiroff, R., & Lopez, B. T. (2023). Direct LiDAR-inertial odometry: Lightweight LIO with continuous-time motion correction. *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 3983–3989. <https://doi.org/10.1109/ICRA48891.2023.10160508>
- Chen, Z., Song, C., Wang, B., Tao, X., Zhang, X., Lin, F., & Cheng, J. C. P. (2025). Automated reality capture for indoor inspection using BIM and a multi-sensor quadruped robot. *Automation in Construction*, 170, 105930. <https://doi.org/10.1016/j.autcon.2024.105930>
- Cordero, A. H. (2024). *Point cloud transport plugins*. [https://github.com/ros-perception/point\\_cloud\\_transport\\_plugins](https://github.com/ros-perception/point_cloud_transport_plugins)
- Cordero, A. H. (2025a). *Image transport plugins*. [https://github.com/ros-perception/image\\_transport\\_plugins](https://github.com/ros-perception/image_transport_plugins)
- Cordero, A. H. (2025b). *Robot state publisher*. [https://github.com/ros/robot\\_state\\_publisher](https://github.com/ros/robot_state_publisher)

- da Silva Lubanco, D. L., Pichler-Scheder, M., & Schlechter, T. (2020). A novel frontier-based exploration algorithm for mobile robots. *2020 6th International Conference on Mechatronics and Robotics Engineering (ICMRE)*, 1–5. <https://doi.org/10.1109/ICMRE49073.2020.9064866>
- Dai, B., Khorrambakht, R., Krishnamurthy, P., & Khorrami, F. (2024). Sailing through point clouds: Safe navigation using point cloud based control barrier functions. *IEEE Robotics and Automation Letters*, 9(9), 7731–7738. <https://doi.org/10.1109/LRA.2024.3431870>
- Durrant-Whyte, H., & Bailey, T. (2006). Simultaneous localization and mapping: Part I. *IEEE Robotics & Automation Magazine*, 13(2), 99–110. <https://doi.org/10.1109/MRA.2006.1638022>
- Fan, Y., Pei, Z., Wang, C., Li, M., Tang, Z., & Liu, Q. (2024). A review of quadruped robots: Structure, control, and autonomous motion. *Advanced Intelligent Systems*, 6(6), 2300783. <https://doi.org/10.1002/aisy.202300783>
- Foote, T. (2013). tf: The transform library. *2013 IEEE Conference on Technologies for Practical Robot Applications (TePRA)*, 1–6. <https://doi.org/10.1109/TePRA.2013.6556373>
- Foxglove Technologies, Inc. (2025). *RViz vs Foxglove*. <https://foxglove.dev/robotics/rviz-vs-foxglove>
- Gan, V. J. L., Hu, D., Wang, Y., & Zhai, R. (2025). Automated indoor 3D scene reconstruction with decoupled mapping using quadruped robot and LiDAR sensor. *Computer-Aided Civil and Infrastructure Engineering*, 1–18. <https://doi.org/10.1111/mice.13450>
- Gan, V. J. L., Hu, D., Zhai, R., & Wang, Y. (2024). Automated as-built 3D reconstruction using quadruped robot and 3D LiDAR sensor. *International Symposium on Automation and Robotics in Construction (ISARC) Proceedings, 2024 Proceedings of the 41st ISARC, Lille, France*, 129–135. <https://doi.org/10.22260/ISARC2024/0018>
- Ge, L., & Sadhu, A. (2025). Deep learning-enhanced smart ground robotic system for automated structural damage inspection and mapping. *Automation in Construction*, 170, 105951. <https://doi.org/10.1016/j.autcon.2024.105951>
- Gojda, M. (2023). The potential of aerial reconnaissance in the detection, mapping and 3D reconstruction modelling of crop-marked military components of Bohemia's postmedieval and early industrial landscape. *Heritage*, 6(4), Article 4. <https://doi.org/10.3390/heritage6040187>

- Grisetti, G., Kümmerle, R., Stachniss, C., & Burgard, W. (2010). A tutorial on graph-based SLAM. *IEEE Intelligent Transportation Systems Magazine*, 2(4), 31–43. <https://doi.org/10.1109/MITS.2010.939925>
- Grisetti, G., Stachniss, C., & Burgard, W. (2007). Improved techniques for grid mapping with Rao-Blackwellized particle filters. *IEEE Transactions on Robotics*, 23(1), 34–46. <https://doi.org/10.1109/TRO.2006.889486>
- Guo, Y., Wang, H., Hu, Q., Liu, H., Liu, L., & Bennamoun, M. (2021). Deep learning for 3D point clouds: A survey. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 43(12), 4338–4364. <https://doi.org/10.1109/TPAMI.2020.3005434>
- Hesai Technology Co., Ltd. (2024). *PandarView 2 User Manual*.
- Hesai Technology Co., Ltd. (2025a). *HesaiLidar ROS 2.0*. [https://github.com/HesaiTechnology/HesaiLidar\\_ROS\\_2.0](https://github.com/HesaiTechnology/HesaiLidar_ROS_2.0)
- Hesai Technology Co., Ltd. (2025b). *HesaiTechnology*. <https://github.com/HesaiTechnology>
- Hesai Technology Co., Ltd. (2025c). *PandarXT 32-Channel Mid-Range Mechanical Lidar User Manual*.
- Hess, W., Kohler, D., Rapp, H., & Andor, D. (2016). Real-time loop closure in 2D LiDAR SLAM. *2016 IEEE International Conference on Robotics and Automation (ICRA)*, 1271–1278. <https://doi.org/10.1109/ICRA.2016.7487258>
- Hu, D., & Gan, V. J. L. (2025). Semantic navigation for automated robotic inspection and indoor environment quality monitoring. *Automation in Construction*, 170, 105949. <https://doi.org/10.1016/j.autcon.2024.105949>
- Hu, D., Gan, V. J. L., & Yin, C. (2023). Robot-assisted mobile scanning for automated 3D reconstruction and point cloud semantic segmentation of building interiors. *Automation in Construction*, 152, 104949. <https://doi.org/10.1016/j.autcon.2023.104949>
- Hu, D., Gan, V. J. L., & Zhai, R. (2024). Automated BIM-to-scan point cloud semantic segmentation using a domain adaptation network with hybrid attention and whitening (DawNet). *Automation in Construction*, 164, 105473. <https://doi.org/10.1016/j.autcon.2024.105473>
- Hu, Q. (2021a). *RandLA-Net*. <https://github.com/QingyongHu/RandLA-Net>

- Hu, Q. (2021b). *SoTA point cloud*. <https://github.com/QingyongHu/SoTA-Point-Cloud>
- Hu, Q., Yang, B., Xie, L., Rosa, S., Guo, Y., Wang, Z., Trigoni, N., & Markham, A. (2020). RandLA-Net: Efficient semantic segmentation of large-scale point clouds. *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 11105–11114. <https://doi.org/10.1109/CVPR42600.2020.01112>
- Hu, X. (2025). *LIO-SAM 6-axis*. [https://github.com/JokerJohn/LIO\\_SAM\\_6AXIS](https://github.com/JokerJohn/LIO_SAM_6AXIS)
- Intel Corporation. (2023). *D435i*. <https://github.com/IntelRealSense/librealsense/blob/master/doc/d435i.md>
- Intel Corporation. (2024). *RealSense ROS*. <https://github.com/IntelRealSense/realsense-ros>
- Intel Corporation. (2025a). *Depth camera D435i*. <https://www.intelrealsense.com/depth-camera-d435i/>
- Intel Corporation. (2025b). *Intel RealSense*. <https://github.com/IntelRealSense>
- Jian, Z., Yan, Z., Lei, X., Lu, Z., Lan, B., Wang, X., & Liang, B. (2023). Dynamic control barrier function-based model predictive control to safety-critical obstacle-avoidance of mobile robot. *2023 IEEE International Conference on Robotics and Automation (ICRA)*, 3679–3685. <https://doi.org/10.1109/ICRA48891.2023.10160857>
- Kang, Z., Yang, J., Yang, Z., & Cheng, S. (2020). A review of techniques for 3D reconstruction of indoor environments. *ISPRS International Journal of Geo-Information*, 9(5), Article 5. <https://doi.org/10.3390/ijgi9050330>
- Kanster. (2018). *Awesome SLAM*. <https://github.com/kanster/awesome-slam>
- KeeganRen. (2018). *Awesome SLAM list*. <https://github.com/OpenSLAM/awesome-SLAM-list>
- Kim, J., Chung, D., Kim, Y., & Kim, H. (2021). Semantic segmentation of 3D point cloud data acquired from robot dog for scaffold monitoring. *International Symposium on Automation and Robotics in Construction (ISARC) Proceedings, 2021 Proceedings of the 38th ISARC, Dubai, UAE*, 784–788. <https://doi.org/10.22260/ISARC2021/0106>

- Kim, J., Chung, D., Kim, Y., & Kim, H. (2022). Deep learning-based 3D reconstruction of scaffolds using a robot dog. *Automation in Construction*, 134, 104092. <https://doi.org/10.1016/j.autcon.2021.104092>
- Kim, T. (2023). *SLAM resources for beginner*. <https://github.com/Taeyoung96/SLAM-Resources-for-Beginner>
- Kitware. (2025). *LidarView*. <https://github.com/Kitware/LidarView>
- Knese, K. (2025). *Rosbag2*. <https://github.com/ros2/rosbag2>
- Kok, M., Solin, A., & Schön, T. B. (2024). Rao-Blackwellized particle smoothing for simultaneous localization and mapping. *Data-Centric Engineering*, 5, e15. <https://doi.org/10.1017/dce.2024.12>
- Krauss, C. D. (2025). *ROS2 camera LiDAR fusion*. [https://github.com/CDonosok/ros2\\_camera\\_lidar\\_fusion](https://github.com/CDonosok/ros2_camera_lidar_fusion)
- Kümmerle, R., Grisetti, G., Strasdat, H., Konolige, K., & Burgard, W. (2011). G2o: A general framework for graph optimization. *2011 IEEE International Conference on Robotics and Automation*, 3607–3613. <https://doi.org/10.1109/ICRA.2011.5979949>
- Labbé, M. (2024). *List of open source SLAM projects*. <https://github.com/introlab/rtabmap/wiki/List-of-Open-Source-SLAM-projects>
- Labbé, M., & Michaud, F. (2014). Online global loop closure detection for large-scale multi-session graph-based SLAM. *2014 IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2661–2666. <https://doi.org/10.1109/IROS.2014.6942926>
- Labbé, M., & Michaud, F. (2019). RTAB-Map as an open-source lidar and visual simultaneous localization and mapping library for large-scale and long-term online operation. *Journal of Field Robotics*, 36(2), 416–446. <https://doi.org/10.1002/rob.21831>
- Lalancette, C. (2024). *Teleop twist keyboard*. [https://github.com/ros2/teleop\\_twist\\_keyboard/pull/29](https://github.com/ros2/teleop_twist_keyboard/pull/29)
- Li, C., Zhang, Y., Yu, Z., Liu, X., & Shi, Q. (2024). A robust visual SLAM system for small-scale quadruped robots in dynamic environments. *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 321–326. <https://doi.org/10.1109/IROS58592.2024.10802163>

- Li, N., Cao, J., & Huang, Y. (2023). Fabrication and testing of the rescue quadruped robot for post-disaster search and rescue operations. *2023 IEEE 3rd International Conference on Electronic Technology, Communication and Information (ICETCI)*, 723–729. <https://doi.org/10.1109/ICETCI57876.2023.10176723>
- Li, Q., Cicirelli, F., Vinci, A., Guerrieri, A., Qi, W., & Fortino, G. (2025). Quadruped robots: Bridging mechanical design, control, and applications. *Robotics*, 14(5), Article 5. <https://doi.org/10.3390/robotics14050057>
- Li, Y., & Shi, C. (2018). Localization and navigation for indoor mobile robot based on ROS. *2018 Chinese Automation Congress (CAC)*, 1135–1139. <https://doi.org/10.1109/CAC.2018.8623225>
- Lin, J., & Zhang, F. (2022). R3LIVE: A robust, real-time, RGB-colored, LiDAR-inertial-visual tightly-coupled state estimation and mapping package. *2022 International Conference on Robotics and Automation (ICRA)*, 10672–10678. <https://doi.org/10.1109/ICRA46639.2022.9811935>
- Lin, T. (2022). *Awesome visual SLAM*. <https://github.com/tzutalin/awesome-visual-slam>
- Lou, L., Li, Y., Zhang, Q., & Wei, H. (2023). SLAM and 3D semantic reconstruction based on the fusion of LiDAR and monocular vision. *Sensors*, 23(3), Article 3. <https://doi.org/10.3390/s23031502>
- Macenski, S. (2025a). *Navigation2*. <https://github.com/ros-navigation/navigation2>
- Macenski, S. (2025b). *Perception PCL*. [https://github.com/ros-perception/perception\\_pcl](https://github.com/ros-perception/perception_pcl)
- Macenski, S. (2025c). *SLAM Toolbox*. [https://github.com/SteveMacenski/slam\\_toolbox](https://github.com/SteveMacenski/slam_toolbox)
- Macenski, S., & Jambrecic, I. (2021). SLAM Toolbox: SLAM for the dynamic world. *Journal of Open Source Software*, 6(61), 2783. <https://doi.org/10.21105/joss.02783>
- Macenski, S., Martín, F., White, R., & Clavero, J. G. (2020). The Marathon 2: A navigation system. *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2718–2725. <https://doi.org/10.1109/IROS45743.2020.9341207>

- Macenski, S., Moore, T., Lu, D., Merzlyakov, A., & Ferguson, M. (2023). From the Desks of ROS Maintainers: A Survey of Modern & Capable Mobile Robotics Algorithms in the Robot Operating System 2. *Robotics and Autonomous Systems*, 168, 104493. <https://doi.org/10.1016/j.robot.2023.104493>
- Naderi, H. (2025, July 2). *Unitree Go2 SLAM Nav2*. <https://github.com/h-naderi/unitree-go2-slam-nav2>
- Natowi. (2023). *3D reconstruction with deep learning methods*. <https://github.com/natowi/3D-Reconstruction-with-Deep-Learning-Methods>
- Okolo, C. F. (2025). *Go2 SLAM 2D 3D*. [https://github.com/felixokolo/go2\\_slam\\_2d\\_3d](https://github.com/felixokolo/go2_slam_2d_3d)
- Open Robotics. (2021). *ROS: Home*. <https://www.ros.org/>
- Pang, C., Zhou, L., & Huang, X. (2024). A low-cost 3D SLAM system integration of autonomous exploration based on Fast-ICP enhanced LiDAR-inertial odometry. *Remote Sensing*, 16(11), Article 11. <https://doi.org/10.3390/rs16111979>
- Paplhham, J. (2019). *Point cloud transport*. [https://github.com/paplhjak/point\\_cloud\\_transport](https://github.com/paplhjak/point_cloud_transport)
- Pepe, M., Alfio, V. S., Costantino, D., & Scaringi, D. (2022). Data for 3D reconstruction and point cloud classification using machine learning in cultural heritage environment. *Data in Brief*, 42, 108250. <https://doi.org/10.1016/j.dib.2022.108250>
- Qiao, J., Guo, J., & Li, Y. (2024). Simultaneous localization and mapping (SLAM)-based robot localization and navigation algorithm. *Applied Water Science*, 14(7), 151. <https://doi.org/10.1007/s13201-024-02183-6>
- Rezende, A. M. C., Júnior, G. P. C., Fernandes, R., Miranda, V. R. F., Azpúrua, H., Pessin, G., & Freitas, G. M. (2020). Indoor localization and navigation control strategies for a mobile robot designed to inspect confined environments. *2020 IEEE 16th International Conference on Automation Science and Engineering (CASE)*, 1427–1433. <https://doi.org/10.1109/CASE48305.2020.9217005>
- Rusinkiewicz, S., & Levoy, M. (2001). Efficient variants of the ICP algorithm. *Proceedings Third International Conference on 3-D Digital Imaging and Modeling*, 145–152. <https://doi.org/10.1109/IM.2001.924423>
- Sasaki, R. (2022). *LIDARSLAM ROS2*. [https://github.com/rsasaki0109/lidarslam\\_ros2](https://github.com/rsasaki0109/lidarslam_ros2)



- Serrano, J. C. M. (2025). *Go2 robot*. [https://github.com/Unitree-Go2-Robot/go2\\_robot](https://github.com/Unitree-Go2-Robot/go2_robot)
- Shan, T. (2023). *LIO-SAM*. <https://github.com/TixiaoShan/LIO-SAM>
- Shan, T., Englot, B., Meyers, D., Wang, W., Ratti, C., & Rus, D. (2020). LIO-SAM: Tightly-coupled LiDAR inertial odometry via smoothing and mapping. *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 5135–5142. <https://doi.org/10.1109/IROS45743.2020.9341176>
- Shan, T., Englot, B., Ratti, C., & Rus, D. (2021). LVI-SAM: Tightly-coupled LiDAR-visual-inertial odometry via smoothing and mapping. *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 5692–5698. <https://doi.org/10.1109/ICRA48506.2021.9561996>
- SilenceOverflow. (2023). *Awesome SLAM*. <https://github.com/SilenceOverflow/Awesome-SLAM>
- Stavrinos, G. (2025). *Odom to tf ROS2*. [https://github.com/gstavrinis/odom\\_to\\_tf\\_ros2](https://github.com/gstavrinis/odom_to_tf_ros2)
- Sun, W., Shen, D., Liu, K., Li, Z., & Zhou, B. (2024). Multi-sensor fusion-based perception and navigation framework for quadruped robots in complex environments\*. *2024 43rd Chinese Control Conference (CCC)*, 4561–4566. <https://doi.org/10.23919/CCC63176.2024.10662127>
- Tanveer, M. H., Fatima, Z., Mariam, H., Rehman, T., & Voicu, R. C. (2024). Three-dimensional outdoor object detection in quadrupedal robots for surveillance navigations. *Actuators*, 13(10), Article 10. <https://doi.org/10.3390/act13100422>
- Torres, M. A. V., Braun, A., & Borrmann, A. (2023). BIM-SLAM: Integrating BIM models in multi-session SLAM for lifelong mapping using 3D LiDAR. *International Symposium on Automation and Robotics in Construction (ISARC) Proceedings, 2023 Proceedings of the 40th ISARC, Chennai, India*, 521–528. <https://doi.org/10.22260/ISARC2023/0070>
- Unitree Robotics. (2023a). *DDS application*. <https://support.unitree.com/home/en/developer/DDS%20Application>
- Unitree Robotics. (2023b). *LiDAR application*. <https://support.unitree.com/home/en/developer/LiDAR%20Application>

Unitree Robotics. (2024a). *Architecture description*.  
<https://support.unitree.com/home/en/developer/Architecture%20Description>

Unitree Robotics. (2024b). *Obtain SDK*.  
<https://support.unitree.com/home/en/developer/Obtain%20SDK>

Unitree Robotics. (2024c). *Odometer services interface*.  
[https://support.unitree.com/home/en/developer/Odometer\\_service](https://support.unitree.com/home/en/developer/Odometer_service)

Unitree Robotics. (2024d). *Point-LIO unilidar*.  
[https://github.com/unitreerobotics/point\\_lio\\_unilidar](https://github.com/unitreerobotics/point_lio_unilidar)

Unitree Robotics. (2024e). *ROS2 services interface*.  
[https://support.unitree.com/home/en/developer/ROS2\\_service](https://support.unitree.com/home/en/developer/ROS2_service)

Unitree Robotics. (2025a). *4D Lidar L1 application scenarios*. <https://www.unitree.com/LiDAR>

Unitree Robotics. (2025b). *About Go2*. <https://support.unitree.com/home/en/developer/>

Unitree Robotics. (2025c). *Expansion dock configuration*.  
[https://support.unitree.com/home/en/developer/module\\_update](https://support.unitree.com/home/en/developer/module_update)

Unitree Robotics. (2025d). *Go2 app download*. <https://www.unitree.com/app/go2>

Unitree Robotics. (2025e). *Go2 description*. [https://github.com/Unitree-Go2-Robot/go2\\_description](https://github.com/Unitree-Go2-Robot/go2_description)

Unitree Robotics. (2025f). *Motion Services Interface V2.0*.  
[https://support.unitree.com/home/en/developer/Motion\\_Services\\_Interface\\_V2.0](https://support.unitree.com/home/en/developer/Motion_Services_Interface_V2.0)

Unitree Robotics. (2025g). *Robot dog B1*. <https://www.unitree.com/b1>

Unitree Robotics. (2025h). *Robot dog B2 inspection robot*. <https://www.unitree.com/b2>

Unitree Robotics. (2025i). *Robot dog Go2 quadruped robot*. <https://www.unitree.com/go2>

Unitree Robotics. (2025j). *SLAM and navigation services interface*.  
[https://support.unitree.com/home/en/developer/SLAM%20and%20Navigation\\_service](https://support.unitree.com/home/en/developer/SLAM%20and%20Navigation_service)

- Unitree Robotics. (2025k). *Unitree 4D LiDAR L2*. <https://www.unitree.com/L2>
- Unitree Robotics. (2025l). *Unitree B2-W*. <https://www.unitree.com/b2-w>
- Unitree Robotics. (2025m). *Unitree Go2-W*. <https://www.unitree.com/go2-w>
- Unitree Robotics. (2025n). *Unitree ROS2*. [https://github.com/unitreerobotics/unitree\\_ros2](https://github.com/unitreerobotics/unitree_ros2)
- Unitree Robotics. (2025o). *Unitree SDK2*. [https://github.com/unitreerobotics/unitree\\_sdk2](https://github.com/unitreerobotics/unitree_sdk2)
- Unitree Robotics. (2025p). *Unitree SDK2 Python*.  
[https://github.com/unitreerobotics/unitree\\_sdk2\\_python](https://github.com/unitreerobotics/unitree_sdk2_python)
- Unlu, H. U., Gonçalves, V. M., Chaikalis, D., Tzes, A., & Khorrami, F. (2024). A control barrier function-based motion planning scheme for a quadruped robot. *2024 IEEE International Conference on Robotics and Automation (ICRA)*, 12172–12178.  
<https://doi.org/10.1109/ICRA57147.2024.10610210>
- Vashisht, A., Gandhi, G. C., Kalra, S., & Saini, D. K. (2025). Hybrid robot navigation: Integrating monocular depth estimation and visual odometry for efficient navigation on low-resource hardware. *Computers and Electrical Engineering*, 124, 110375.  
<https://doi.org/10.1016/j.compeleceng.2025.110375>
- Verifiable & Control-Theoretic Robotics Laboratory. (2024). *Direct LiDAR inertial odometry*.  
[https://github.com/vectr-ucla/direct\\_lidar\\_inertial\\_odometry](https://github.com/vectr-ucla/direct_lidar_inertial_odometry)
- Wang, X., Liu, J., Zhang, T., & Zhang, J. (2024). Applied research of agricultural quadruped robots. *2024 IEEE 13th International Conference on Communication Systems and Network Technologies (CSNT)*, 954–957.  
<https://doi.org/10.1109/CSNT60213.2024.10545756>
- Xu, W., & Zhang, F. (2021). FAST-LIO: A fast, robust LiDAR-inertial odometry package by tightly-coupled iterated Kalman filter. *IEEE Robotics and Automation Letters*, 6(2), 3317–3324. <https://doi.org/10.1109/LRA.2021.3064227>
- Xu, Y., Li, B., Yang, S., Zhi, Y., Li, R., & Chen, T. (2024). MSOT-Q: Multi-sensor fusion SLAM with object detection tracking for quadruped robots. *2024 36th Chinese Control and Decision Conference (CCDC)*, 4485–4492.  
<https://doi.org/10.1109/CCDC62350.2024.10587825>

- Xue, J., Hou, X., & Zeng, Y. (2021). Review of image-based 3D reconstruction of building for automated construction progress monitoring. *Applied Sciences*, 11(17), Article 17. <https://doi.org/10.3390/app11177840>
- Yamauchi, B. (1997). A frontier-based approach for autonomous exploration. *Proceedings 1997 IEEE International Symposium on Computational Intelligence in Robotics and Automation CIRA'97. "Towards New Computational Principles for Robotics and Automation,"* 146–151. <https://doi.org/10.1109/CIRA.1997.613851>
- Yarovoi, A., & Cho, Y. K. (2024). Review of simultaneous localization and mapping (SLAM) for construction robotics applications. *Automation in Construction*, 162, 105344. <https://doi.org/10.1016/j.autcon.2024.105344>
- Yin, C. (2024). *ResPointNet2*. <https://github.com/PointCloudYC/ResPointNet2>
- Yin, C., Wang, B., Gan, V. J. L., Wang, M., & Cheng, J. C. P. (2021). Automated semantic segmentation of industrial point clouds using ResPointNet++. *Automation in Construction*, 130, 103874. <https://doi.org/10.1016/j.autcon.2021.103874>
- Zeren, B. (2023). *LIO-SAM-Color*. <https://github.com/leo-drive/LIO-SAM-COLOR/tree/master>
- Zhai, R., Zou, J., Gan, V. J. L., Han, X., Wang, Y., & Zhao, Y. (2024). Semantic enrichment of BIM with IndoorGML for quadruped robot navigation and automated 3D scanning. *Automation in Construction*, 166, 105605. <https://doi.org/10.1016/j.autcon.2024.105605>
- Zhang, H., Li, L., Jiang, A., Xu, J., Shen, H., Li, Y., Ji, A., & Wang, Z. (2025). LICFM3-SLAM: LiDAR-inertial-camera fusion and multimodal multilevel matching for bionic quadruped inspection robot mapping. *IEEE Transactions on Instrumentation and Measurement*, 74, 1–17. <https://doi.org/10.1109/TIM.2025.3555702>
- Zhang, J. (2025). *Autonomy stack Go2*. [https://github.com/jizhang-cmu/autonomy\\_stack\\_go2](https://github.com/jizhang-cmu/autonomy_stack_go2)
- Zhang, J., & Singh, S. (2014). LOAM: LiDAR odometry and mapping in real-time. *Robotics: Science and Systems Conference (RSS)*, 109–111.
- Zhu, J., Li, H., & Zhang, T. (2024). Camera, LiDAR, and IMU based multi-sensor fusion SLAM: A survey. *Tsinghua Science and Technology*, 29(2), 415–429. <https://doi.org/10.26599/TST.2023.9010010>